



ARTIFICIAL INTELLIGENCE AND DATA ENGINEERING
project report for the examination
of
Large Scale and Multi-Structured Data Bases

Student:
Alessandro Diana

AA. 2024/2025

Contents

1	Introduction	1
1.1	Overview of the MyFuture Application	1
2	Requirements Analysis	2
2.1	System Objectives and Core Features	2
2.2	Main Actors and Roles	2
2.3	Functional Requirements	2
2.3.1	Unregistered users	2
2.3.2	Registered Users	3
2.3.3	Admin	3
2.4	Non-functional Requirements	4
3	System Design and Use Cases	5
3.1	UML Use Case Diagram	5
3.2	UML Class Diagram	6
3.3	Mock-ups of Main Pages	7
3.3.1	Unregistered Users	7
3.3.2	Registered Users	8
3.3.3	Admin	13
4	Dataset and Data Modeling	14
4.1	Variety and Sources of Financial Data	14
4.2	Dataset Description	14
4.3	Data Preprocessing and Integration	15
4.3.1	Transaction Generation Pipeline	16
5	MongoDB Relations Designs	17
5.1	MongoDB Collections	17
5.2	App and Dataset reminder	18
5.3	Assumptions	18
5.3.1	Data Access Patterns and Workload Estimation	18
5.3.2	Storage Constraints and Primitive Types in MongoDB	19
5.4	Real Data Vision	19
5.4.1	Asset and Transactions volumes	19
5.4.2	User Profiles and Behavioral Workload Analysis	20
5.4.3	Asset Identification and Symbol usage	21
5.5	Relations	21
5.5.1	User ↔ Asset Relationship (Portfolio) – Choice: Embedding	22
5.5.2	User/Asset ↔ Transaction Relationship – Choice: Linking with Extended Reference	23
5.5.3	Asset ↔ Asset.Prices Relationship – Choice: Time-Series Pattern	27

5.6	Experimental Validation: Performance Benchmarking	28
5.6.1	Benchmark Objectives	28
5.6.2	Analysis of Results for Users and Transactions	29
5.6.3	Analysis of Results for Assets and Assets_prices	31
5.7	Synthesis of Architectural Decisions	31
5.8	Conclusion	32
6	MongoDB Document Schemas	33
6.1	User Collection	33
6.2	Asset Collection	34
6.3	Transaction Collection	35
6.4	Asset Prices (Time-Series Data)	36
6.5	News Collection	37
6.6	Counters Collection	37
7	MongoDB Indexing Strategy and Performance Validation	38
7.0.1	User Collection	38
7.0.2	Asset Prices	39
7.0.3	Transactions	40
7.0.4	News	41
7.1	Benchmark Conclusions	42
8	MongoDB replica and sharding	43
8.1	Replica set	43
8.2	Sharding	43
8.2.1	General Principles	43
8.2.2	Asset Prices: Optimization for Time-Series Analysis	44
8.2.3	Transactions: User-Centric Partitioning	44
8.2.4	News: Strategic Outlook	44
8.2.5	Users and Assets: Metadata Stability	45
9	Redis Design	46
9.1	Redis Architectural Role	46
9.2	Redis Key-Value Structure and Data Mapping	46
9.2.1	User key	46
9.2.2	Asset and Market Data	46
9.2.3	News	47
9.3	Synchronization and Persistence Policies	47
9.4	Redis Replication and Fault Tolerance Strategy	47
10	Database Operations and Queries	49
10.1	CRUD Operations	49
10.2	Advanced Queries and Aggregations	49
10.2.1	User	51
10.2.2	Asset	52
10.2.3	Asset_prices	53
10.2.4	Transaction	56
10.2.5	News	58
10.3	Operation interacting with Redis Caching	58
10.3.1	Read Optimization (Caching Strategy)	59
10.3.2	Write-Through and Cache Invalidation	59
11	Considerations on CAP Theorem	60
11.1	Emphasis on Consistency in the System's Architecture	60
11.2	Scalability and High Availability Strategy	60

11.3	Balancing Consistency and Availability: The Hybrid Model	60
11.4	Design for Future Read-Write Splitting	61
12	Implementation Details	62
12.1	Frameworks and Technologies	62
12.2	Project Structure and Packages	62
13	Use of Artificial Intelligence Tools	65
13.1	Purpose	65
13.2	How	65
13.3	Critical Evaluation	65
14	Future Improvements and Extensions	65
14.1	Machine Learning for Financial Insights	65
14.2	Enhanced Security and Multi-Factor Authentication (MFA)	66
14.3	Horizontal Scaling and Microservices Decomposition	66
14.4	Granular Read Preference Optimization	66

List of Figures

1	Image showing the use cases diagram for the system.	5
2	Image showing the UML class diagram for the system.	6
3	Image showing the home screen for unregistered users.	7
4	Image showing the news screen for unregistered users.	7
5	Image showing the asset list screen for unregistered users.	8
6	Image showing the home screen for registered users.	9
7	Image showing the plot screen of an assets for registered users.	9
8	Image showing the assets list for registered users.	10
9	Image showing the table with the best assets of the day for registered users. . . .	10
10	Image showing the table with the worst assets of the day for registered users. . .	11
11	Image showing the table with the most traded assets of the day for registered users.	11
12	Image showing the list of latest news for registered users.	12
13	Image showing the open news for registered users.	12
14	Image showing the home page for admin.	13
15	Image depicting data on some of the major assets on the NYSE stock exchange.	19
16	Image depicting data on some of the major assets on the NASDAQ stock exchange.	20
17	The image illustrates the polymorphic structure of the recentTransactions array. It highlights the field differentiation between financial trades (requiring symbol and quantity) and account movements (deposits/withdrawals).	25
18	Image showing the size of the data in the dataset and the size of the embedded arrays for users, depending on architectural choices.	28
19	Image showing the times in milliseconds for transaction recovery in various time frames based on the technique used and the time saved by using the technique I chose.	29
20	Image showing benchmarks for assets and assets_prices cases.	30
21	Detailed view of a MongoDB User document.	33
22	Detailed view of a MongoDB User document.	34
23	Example of documents from the assets collection.	35
24	Detailed view of a MongoDB User document.	35
25	Example of documents from the asset_prices collection.	36
26	Example of documents from the news collection.	36
27	View of the documents from the counters collection.	37
28	Image showing index benchmark results for users collection.	38
29	Image showing index benchmark results for asset_prices collection.	39
30	Image showing index benchmark results for transactions collection (filtered by user).	40
31	Image showing index benchmark results for transactions collection (filtered by type).	40
32	Image showing index benchmark results for transactions collection (filtered by status).	41

33	Image showing index benchmark results for news collection.	42
34	Image showing the code for the 10 users with the largest portfolios in terms of different assets.	49
35	Image showing the code for the 10 users with the largest amount of a given asset in their portfolio.	50
36	Image showing the code for viewing the average, minimum, and maximum number of distinct assets held by users.	50
37	Image showing the code for the number of assets by type (share / ETF / crypto).	51
38	Image showing the code for the top 10 sectors by number of listed shares.	51
39	Image showing the code for viewing the top 10 assets with the best growth/worst decline over the time window.	52
40	Image showing the code for the 10 asset have consistently risen/fell over the past week.	53
41	Image showing the code for the top 10 most traded assets on the platform over the time window (day, week, month, ...).	54
42	Image showing the code for analyzing the number of transactions, by categories or by payment type, over time.	54
43	Image showing the code for viewing the total amount of money invested in assets over a specific time range (e.g, week, month, ...).	55
44	Image showing the code for viewing the Top-K users with the highest/worst net financial flow generated by buy and sell operations within a selected time window (e.g., week, month, ...).	55
45	Image showing the code for getting the total monetary volume (sum of totalPrice) for each asset traded during the current day.	56
46	Image showing the code to see the number of news per sector in a time window.	57
47	Image showing the code for identifying the top 5 companies that have been mentioned in the highest number of news articles within a given time window.	57
48	Architecture Overview: High-level view of the backend components.	64

1 Introduction

1.1 Overview of the MyFuture Application

Imagine this: you have finally put aside some money and want to invest a part of it in the free market. You start looking for stocks on Google and, well... you are overwhelmed by the amount of information there is. But okay, you kind of manage and understand the concepts of stocks, ETFs, and so on. But now you want to invest, sure... but how? MyFuture is the app that allows you to invest whatever you want most simply. In other words, free market for dummies.

Moving from the user perspective to its technical nature, MyFuture is a web-based trading application designed to simulate the behavior of a modern online trading platform, providing users with access to a curated set of financial assets, including stocks, Exchange Traded Funds (ETFs), and cryptocurrencies. The application allows users to manage a personal account and a virtual investment portfolio, enabling operations such as viewing asset prices, buying and selling financial instruments, monitoring portfolio performance, and consulting financial news.

Its primary objective is to reproduce, as realistically as possible, the dynamics of a financial market environment while maintaining a simplified and accessible user experience. The application targets users with limited financial expertise, offering an intuitive interface that abstracts the complexity of real-world trading systems while preserving their core mechanisms.

In addition to trading functionalities, MyFuture integrates a financial news section, allowing users to contextualize asset price movements within broader market events. From a technical perspective, the application is built on a distributed data architecture combining MongoDB for persistent storage and Redis for high-performance, real-time data access.

All the developed code and documentation can be found at this link:
https://github.com/Arzazrel/Project_LSMSDB_25.

2 Requirements Analysis

2.1 System Objectives and Core Features

The main objectives of the MyFuture system are:

- To support user account management, including registration, authentication, and role differentiation (regular users and administrators);
- To provide access to a diverse set of financial assets with historical price data;
- To enable buy and sell operations with realistic constraints (market hours, price determination, balance checks);
- Maintain and visualize a user portfolio, including cash balance, asset holdings, and performance indicators;
- Integrate financial news as contextual information for market movements;
- To support analytical and administrative queries, such as rankings, trends, and aggregated statistics.

Core features include:

- Portfolio management with profit and loss computation;
- Historical and near real-time price visualization;
- Transaction history and activity tracking;
- Admin-oriented analytical dashboards based on aggregation queries;
- A hybrid data architecture that leverages MongoDB and Redis to balance durability and performance.

2.2 Main Actors and Roles

The main actors in our application:

- Unregistered Users: can browse the platform and access basic features. However, they cannot perform trading actions or manage a personal portfolio;
- Registered Users: have full access to the trading functionalities. They can view detailed asset data, buy and sell financial assets, manage their portfolio, and track their account balance within the platform;
- Admins: are responsible for managing the platform's data and users.

2.3 Functional Requirements

2.3.1 Unregistered users

- Register within the application;
- Browse through assets;
- Browse through daily financial news (only title and summary);
- View basic information about the platform and how it works.

2.3.2 Registered Users

- Log in and out of the application;
- View his/her personal page containing account information;
- View his/her wallet/account liquidity;
- Credit money to the account;
- Withdraw money from his/her account;
- Views his/her assets with information like: average purchase value, quantity, current price, current gain or loss;
- View the price chart of a personal asset;
- Buy or sell an asset;
- Browse through assets;
- Views the plot of an asset;
- Browse through the news;
- View only the news belonging to a selected category;
- Select a news article and visualize the full text;
- View basic information about assets and how it works (information and instruction page).

2.3.3 Admin

- Log in and out of the application;
- View a list of registered users (with information like username, registration date);
- Suspend/unsuspend user accounts (case of suspicious behavior or violations);
- Browse assets like registered users (access additional views for deleted assets/news);
- Add, update, or remove an asset;
- Browse and visualize news like registered users (more can see deleted news);
- Add, update, or remove a news;
- Calculate statistics (e.g., average number of transactions, number of daily deposits and withdrawals, Total number of registered, suspended, or unsuspended users, ...);

2.4 Non-functional Requirements

- The system must be a website application;
- The system must encrypt users' passwords;
- The system must ensure high availability;
- The system must be intuitive to use for users without any specific training;
- The system must ensure data durability and consistency even in the case of partial failures;
- The system must provide near-real-time updates on asset prices and ensure low-latency access to assets list and last financial news.

3 System Design and Use Cases

3.1 UML Use Case Diagram

The following Use Case Diagram 1 illustrates the interactions between the actors (unregistered/registered Users and Administrators) and the core functionalities of the system.

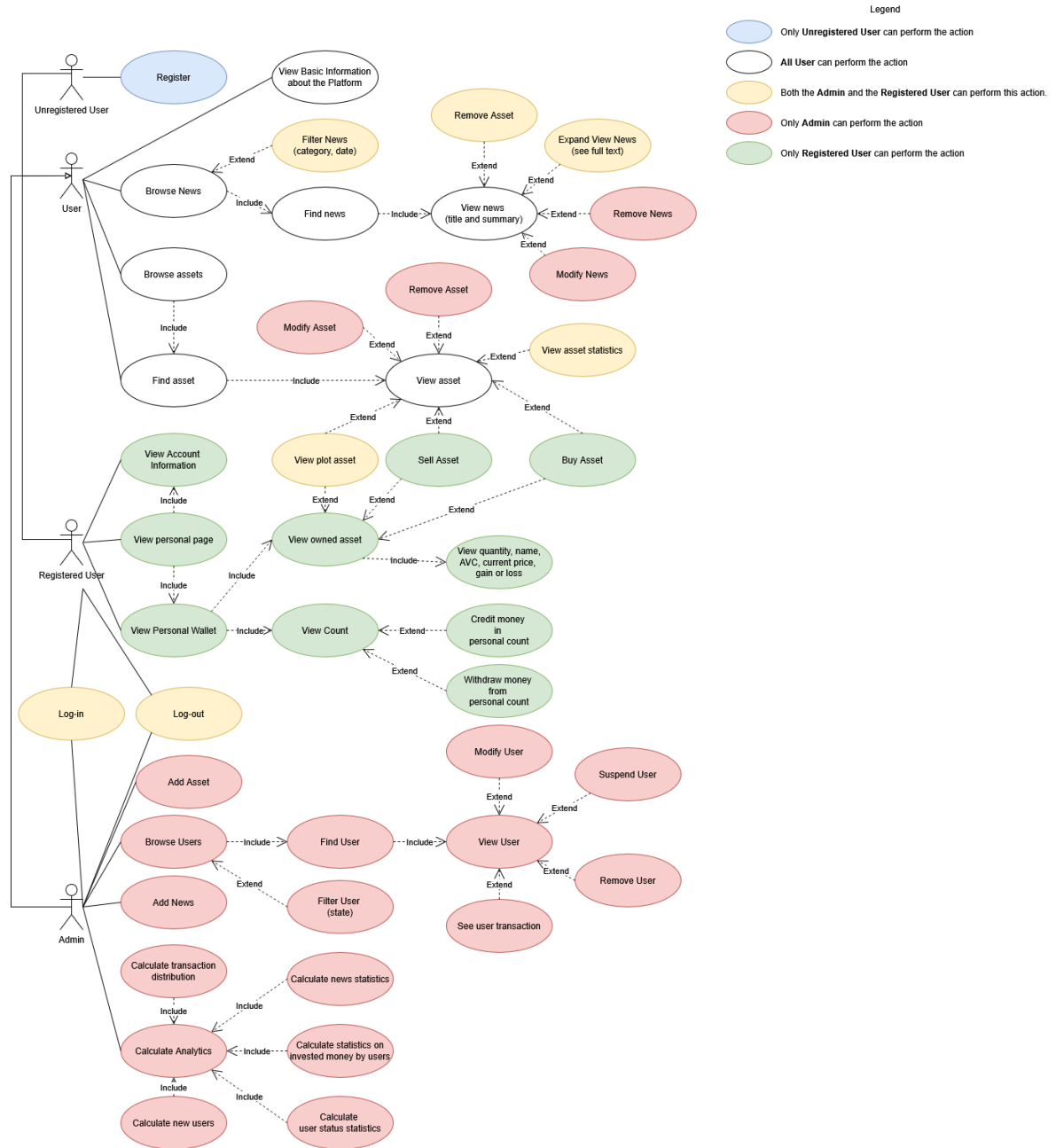


Figure 1: Image showing the use cases diagram for the system.

3.2 UML Class Diagram

This UML class diagram 2 describes the static architecture of the backend, detailing the organization of data and the interaction logic between entities, DTOs, and business services. By modeling the system according to object-oriented design paradigms, it highlights the relationships, hierarchies, and responsibilities of each software component.

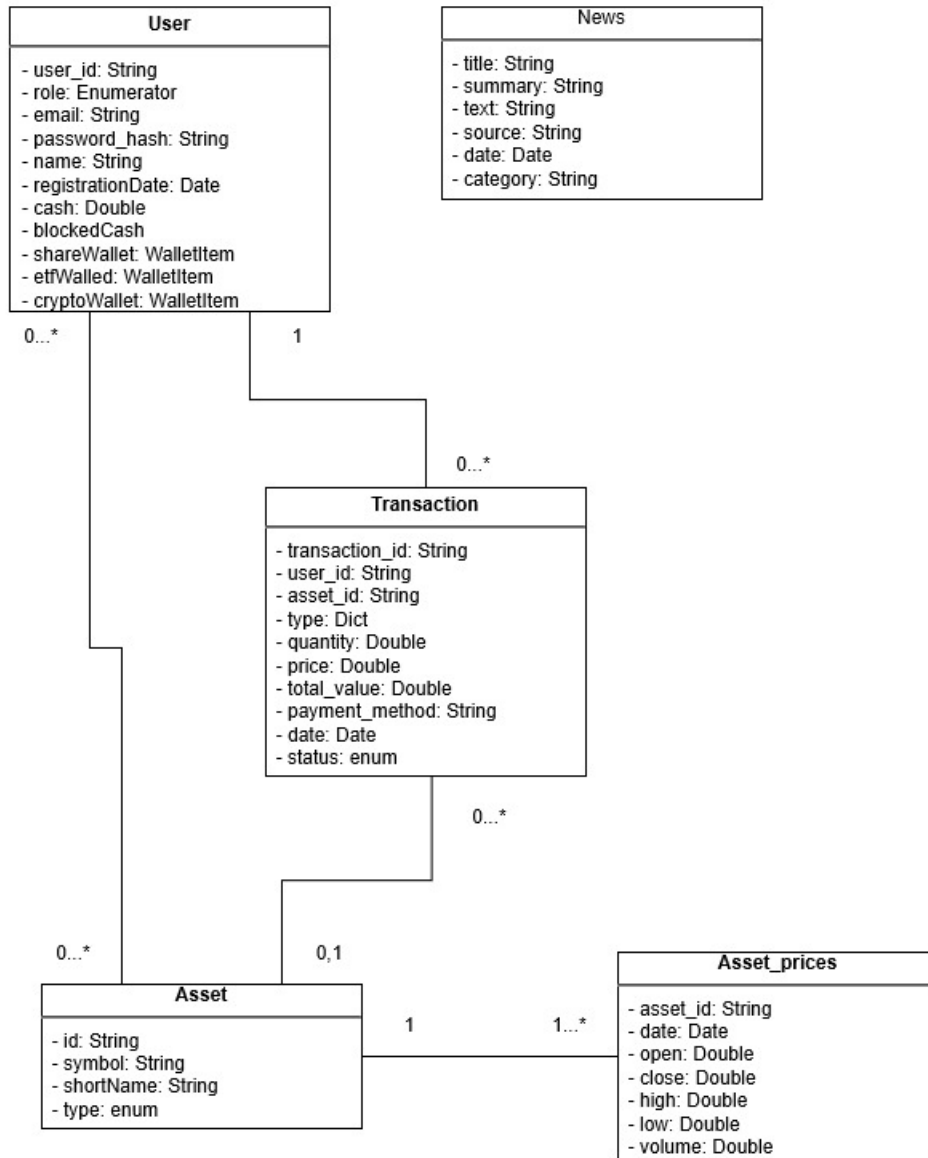


Figure 2: Image showing the UML class diagram for the system.

3.3 Mock-ups of Main Pages

3.3.1 Unregistered Users

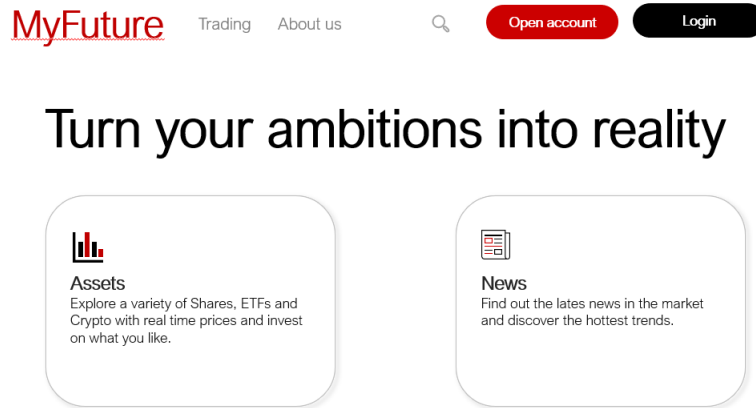


Figure 3: Image showing the home screen for unregistered users.

When you first open the site, you are welcomed by the following homepage 3. Here you have our slogan, and you can find different clickable buttons that allow you to Open an account if don't have one, or log in if you have one, or go to the Assets, Etf's and Cryptos that are available in the current market, as well as the news button to get the latest trends and news in the market. Also, you can do specific researches by clicking on the lens, or you can explore more by clicking on the drop-down menu.

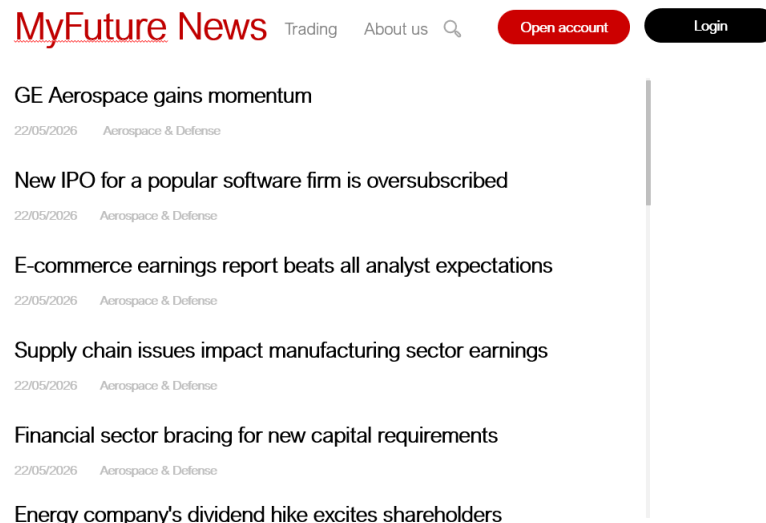
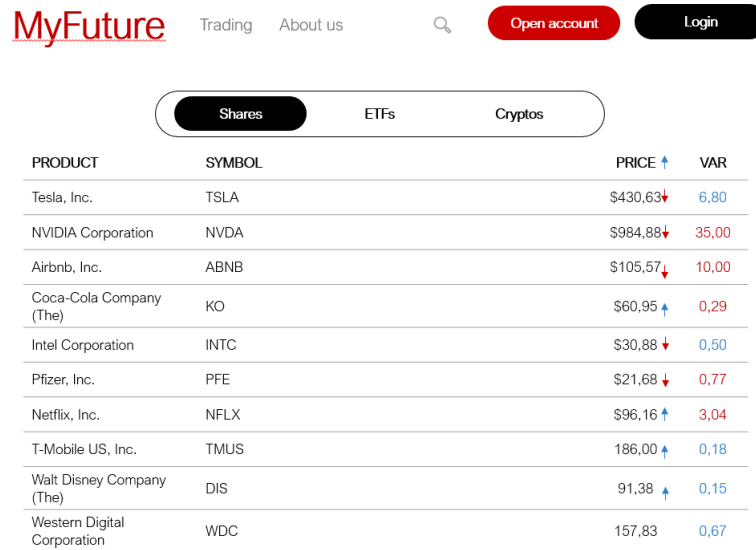


Figure 4: Image showing the news screen for unregistered users.

By clicking on the news button, the unregistered users can see the list of the 10 latest news, but cannot open them unless it signs in, as shown in image 4.



PRODUCT	SYMBOL	PRICE	VAR
Tesla, Inc.	TSLA	\$430,63↓	6,80
NVIDIA Corporation	NVDA	\$984,88↓	35,00
Airbnb, Inc.	ABNB	\$105,57↓	10,00
Coca-Cola Company (The)	KO	\$60,95↑	0,29
Intel Corporation	INTC	\$30,88↓	0,50
Pfizer, Inc.	PFE	\$21,68↓	0,77
Netflix, Inc.	NFLX	\$96,16↑	3,04
T-Mobile US, Inc.	TMUS	186,00↑	0,18
Walt Disney Company (The)	DIS	91,38↑	0,15
Western Digital Corporation	WDC	157,83	0,67

Figure 5: Image showing the asset list screen for unregistered users.

If you click on the assets button, you can find the list of assets with prices that change in real time. On the part on top of the table, you can choose the type of asset you want to see with a clickable button, as shown in image 5.

3.3.2 Registered Users

This shows the personal home page, image 6, when you press the account button on the top right and logging. Here on the top part, there are the two icons of news and assets that allow you to access the news and asset functionalities. You can also find the Account overview, which contains the Total cash you have, the amount you invested, the amount left, and the total price gain/loss. On the center and lower part there are the tables of the products that have been bought. In this example, we have Shares and ETFs. This table shows the quantity that has been bought, the price at the time of purchase, the total value, the BEP, and the P/L%. In the product column, next to the product name, there are three clickable buttons, which are quick access buttons that allow you to buy, sell, or show the price variation in a line plot.

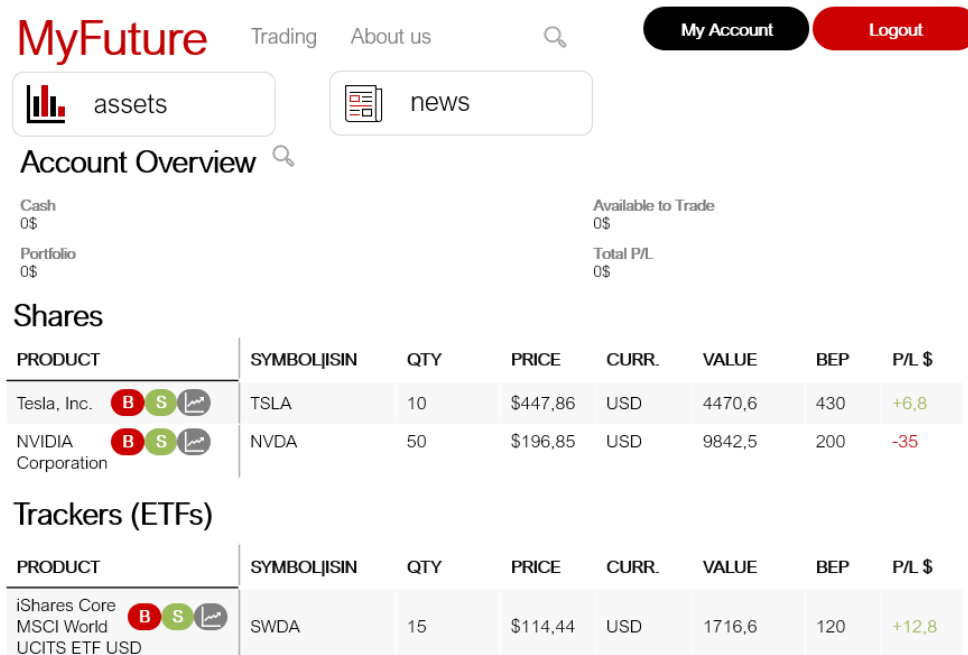


Figure 6: Image showing the home screen for registered users.

The image 7 shows the screen that appears when you press the plot button for a specific asset.

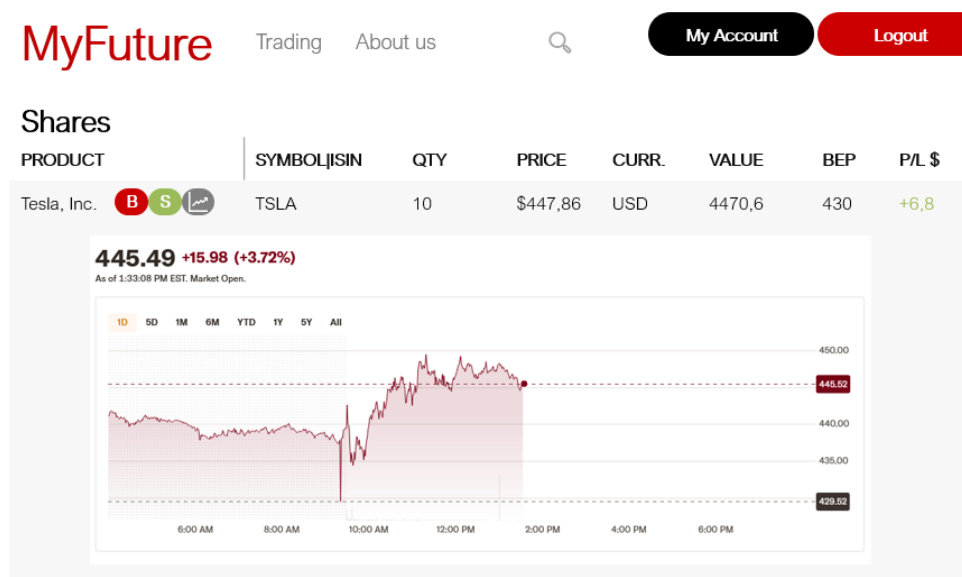


Figure 7: Image showing the plot screen of an assets for registered users.

MyFuture Trading About us Open account Login			
Most Lowest Best			
Shares ETFs Cryptos			
PRODUCT	SYMBOL	PRICE ↑	VAR
Tesla, Inc.	TSLA	\$430,63 ↓	6,80
NVIDIA Corporation	NVDA	\$984,88 ↓	35,00
Airbnb, Inc.	ABNB	\$105,57 ↓	10,00
Coca-Cola Company (The)	KO	\$60,95 ↑	0,29
Intel Corporation	INTC	\$30,88 ↓	0,50
Pfizer, Inc.	PFE	\$21,68 ↓	0,77
Netflix, Inc.	NFLX	\$96,16 ↑	3,04
T-Mobile US, Inc.	TMUS	186,00 ↑	0,18
Walt Disney Company (The)	DIS	91,38 ↑	0,15
Western Digital Corporation	WDC	157,83	0,67

Figure 8: Image showing the assets list for registered users.

By clicking in the asset button as a registered user, you are brought to the asset table, image 8, where you can choose the most trending products that are hyped at the moment, but you can also select the products with the lowest or best change of the day. It also has fast buttons to buy, sell, or see the trend plot, as shown in images 91011.

MyFuture Trading About us My Account Logout				
Most Lowest Best				
PRODUCT	SYMBOL	PRICE	CHANGE	ACTION
Sandisk Corporation	SNDK	\$39,50	20,80 ↑	B S
Fortuna Mining Corp	FSM	\$12,27	15,70 ↑	B S
Alphabet Inc.	GOOG	\$105,57	12,30 ↑	B S
Meta Platforms, Inc.	META	\$60,95	11,39 ↑	B S
Taiwan Semiconduct..	TSM	\$30,88	10,88 ↑	B S
Ring Central, Inc	RNG	\$21,68	10,77 ↑	B S
Analog Device	ADI	\$96,16	9,04 ↑	B S
Corning Incorporate	GLW	186,00	8,18 ↑	B S
AppLovin Corporation	APP	91,38	8,15 ↑	B S
Amazon.com, Inc.	AMZ	157,83	7,67 ↑	B S

Figure 9: Image showing the table with the best assets of the day for registered users.

MyFuture Trading About us  My Account Logout

Most **Lowest** Best

PRODUCT	SYMBOL	PRICE	CHANGE	ACTION
CrowdStrike Holdings	CRWD	\$430,63	33,54 ↓	B S 
Cloudflare, Inc.	NET	\$984,88	15,50 ↓	B S 
jFrog Ltd.	AKAM	\$105,57	15,42 ↓	B S 
Bloom Energy Corporation	FROG	\$60,95	12,54 ↓	B S 
Nebius Group N.V.	BE	\$30,88	11,54 ↓	B S 
Oracle Corporation	NBIS	\$21,68	9,69 ↓	B S 
CoreWave, Inc.	ORCL	\$96,16	8,46 ↓	B S 
Okta, Inc.	CRWV	186,00	7,89 ↓	B S 
Snowflake	SNOW	91,38	7,51 ↓	B S 
Ares Management Corporation	ARES	157,83	6,70 ↓	B S 

Figure 10: Image showing the table with the worst assets of the day for registered users.

MyFuture Trading About us  Open account Login

Most **Lowest** Best

Shares **ETFs** Cryptos

PRODUCT	SYMBOL	PRICE ↑	VAR
Tesla, Inc.	TSLA	\$430,63 ↓	6,80
NVIDIA Corporation	NVDA	\$984,88 ↓	35,00
Airbnb, Inc.	ABNB	\$105,57 ↓	10,00
Coca-Cola Company (The)	KO	\$60,95 ↑	0,29
Intel Corporation	INTC	\$30,88 ↓	0,50
Pfizer, Inc.	PFE	\$21,68 ↓	0,77
Netflix, Inc.	NFLX	\$96,16 ↑	3,04
T-Mobile US, Inc.	TMUS	186,00 ↑	0,18
Walt Disney Company (The)	DIS	91,38 ↑	0,15
Western Digital Corporation	WDC	157,83	0,67

Figure 11: Image showing the table with the most traded assets of the day for registered users.

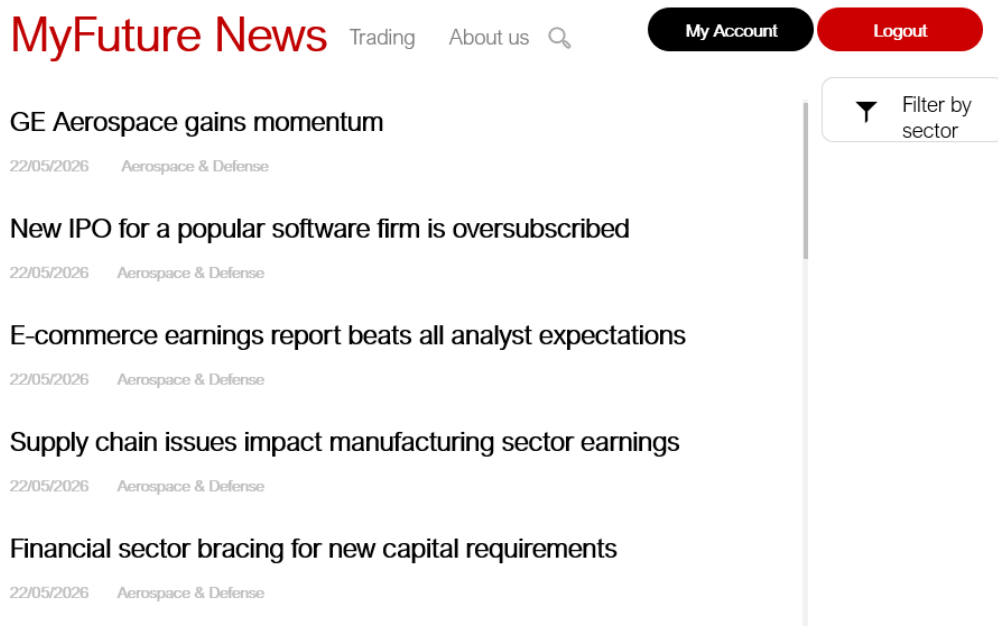


Figure 12: Image showing the list of latest news for registered users.

If you click on the news button when the user is logged in. Here you can see the list of the 10 latest news, filter them by sector, as shown in image 12 (for example Aerospace and Defence), and can open the news of interest. A registered user can click on the news to expand and see all the content, as shown in image 13.

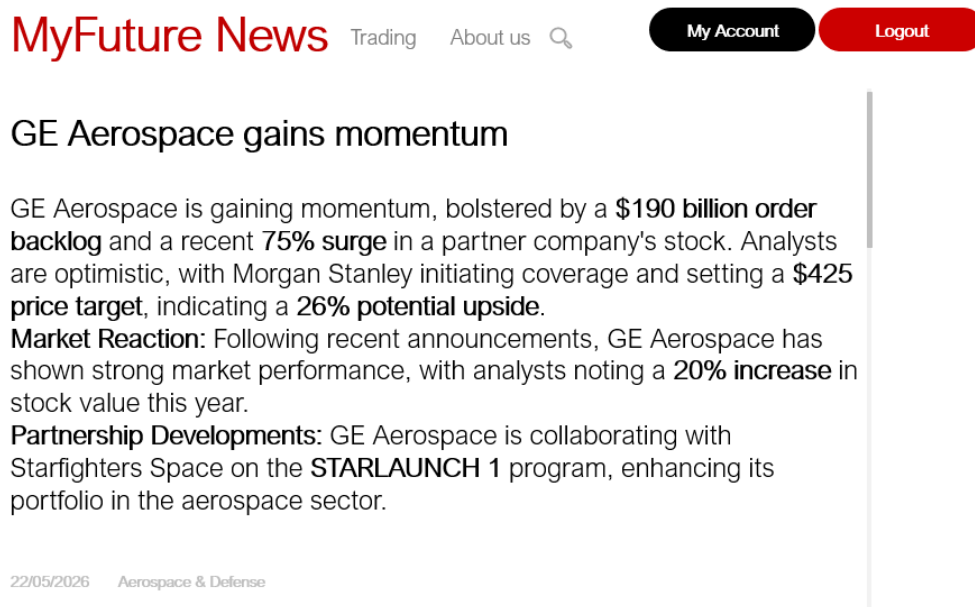


Figure 13: Image showing the open news for registered users.

3.3.3 Admin

This image 14 shows the admin page, where the Admin has access to exclusive controls, such as calculating statistics of transactions, news, and users. I also can show the list of users and suspend an account in case of suspicious behavior. It can also manually add/ remove and modify assets.

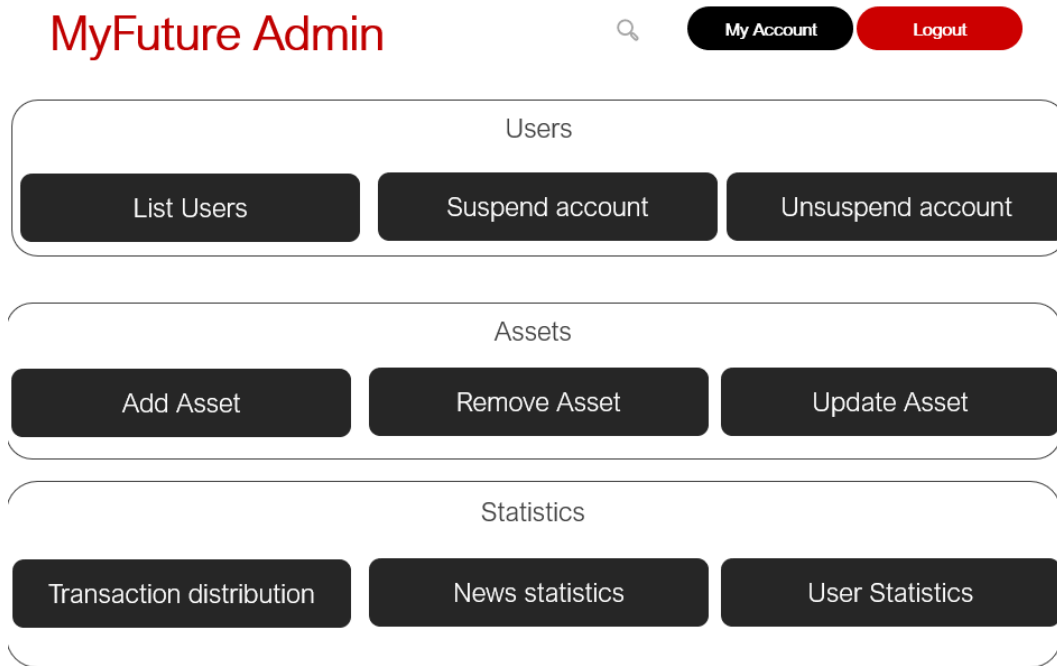


Figure 14: Image showing the home page for admin.

4 Dataset and Data Modeling

4.1 Variety and Sources of Financial Data

A key characteristic of the project is the variety of data sources and data types, which reflects the heterogeneity of real-world financial systems.

The heterogeneity addressed in this project is:

- Time-series data: daily historical prices for financial assets are characterized by high volume and strong temporal ordering;
- Event-based data: transactions represent discrete, immutable financial events with temporal and monetary attributes;
- Textual and semi-structured data: financial news articles include free text, summaries, categorical labels, and references to companies and indices.

The datasets originate from:

- Yahoo Finance is a widely used and reliable financial data provider that is used to retrieve the historical data for the assets.
- Kaggle, which offers publicly available datasets suitable for large-scale data analysis and experimentation. It is used to obtain datasets relating to news and transactions.
- Lists available free on the internet (finance websites and Wikipedia) of companies included in the assets considered for the project, or of the largest companies per continent.

The integration of these heterogeneous sources enables both operational features (trading simulation) and analytical use cases (trend analysis, rankings, and aggregations).

4.2 Dataset Description

The datasets and data collected used in the project are as follows:

- Cryptocurrencies, 74 assets, collected via scripts with Yahoo Finance (13,8 MB);
- Share (collected via scripts with Yahoo Finance, 1542 assets, 928 MB);
 - Top euro companies, 41 assets (26 MB);
 - S&P 500, 501 assets (363 MB)
 - S&P 400, 400 assets (225 MB)
 - S&P 600, 599 assets (314 MB)
- ETF, 50 assets, collected via scripts with Yahoo Finance (23.5 MB);
- User transactions (39222 transactions, 1,45 MB).
<https://www.kaggle.com/datasets/mahmoudshaheen1134/trades-dataset> .
- News (5263 news, 9.88 MB)
 - 3024 news (2.1 MB)
<https://www.kaggle.com/datasets/pratyushpuri/financial-news-market-events-dataset-2025>.

- 400 news (1 MB)
<https://www.kaggle.com/datasets/aravsood7/sentiment-analysis-labelled-financial-news-data>.
- o 1839 news (6.78 MB)
<https://www.kaggle.com/datasets/sayelabualigah/high-quality-financial-news-dataset-for-nlp-tasks?select=dataset.csv>.

This data is saved in the 'dataset' folder of the GitHub project.

4.3 Data Preprocessing and Integration

Significant preprocessing was required to integrate the datasets into a coherent system.

- Asset price data were normalized and imported into MongoDB as daily records, ensuring consistent schemas across different asset types;
- Financial news from multiple sources was cleaned, unified, and projected onto a common schema to guarantee structural uniformity;
- The transaction dataset was not used directly; instead, each record was interpreted as a template to generate a synthetic transaction consistent with the platform's rules.

A significant preprocessing effort was dedicated to the transaction dataset to adapt it for the MyFuture ecosystem. The original dataset consists of 39,221 samples, which were leveraged to populate the historical transaction records in MongoDB.

The source data provided the following attributes:

- `accountAgeDays`: Represents the account age in days at the time of purchase (1,999 distinct values, ranging from 1 to 2,000);
- `numItems`: Indicates the quantity of assets purchased (ranging from 1 to 29);
- `localTime`: A numerical range (0.42 to 5.41) representing isolated time segments rather than full dates or timestamps;
- `paymentMethod`: Categorized into PayPal (9,303 cases), Credit Card (28,004 cases), and Store Credit (1,914 cases);
- `paymentMethodAge`: statistical indicators not relevant to the trading simulation logic;
- `Label`: statistical indicators not relevant to the trading simulation logic.

To align this data with the application's requirements, the following mapping strategy was implemented:

- **User Identification**: The `accountAgeDays` field is mapped as a unique `user_id`. Consequently, 1,999 distinct user profiles were generated, grouping transactions by account age.
- **Asset Quantities**: The `numItems` field defines the volume for a specific transaction. Each transaction is limited to a single asset type.
- **Temporal Normalization**: Since `localTime` and `paymentMethodAge` were incompatible with a real-world calendar, transaction dates are stochastically generated within the valid historical range of the selected asset;
- `paymentMethod`: is used as is;
- `label`: not used.

4.3.1 Transaction Generation Pipeline

For each sample in the dataset, a transaction is synthesized following these procedural steps:

- **Asset Selection:** A financial asset is randomly selected from the global pool (Stocks, ETFs, or Cryptocurrencies).
- **Date and Price Determination:** A valid trading date and time are generated within market hours. The execution price is derived from MongoDB historical data based on the following heuristics:
 - **Opening Price:** Applied if the transaction occurs within one hour of market open;
 - **Closing Price:** Applied if the transaction occurs within one hour of market close;
 - **Intraday Randomization:** A random price point for that day is calculated for all other timeframes.
- **Financial Flow:** The total cost is calculated ($\text{Unit Price} \times \text{Quantity}$). If the payment method is Store Credit, a preliminary "Deposit" transaction is automatically generated 5 minutes before the purchase to ensure sufficient account liquidity.
- **System Integration:** Transactions are persisted in the Document Database (MongoDB), concurrently updating the user's portfolio fields (balance and holdings);
- **Simulated Selling Behavior:** To simulate realistic trading patterns, sell transactions are modeled as a stochastic consequence of purchase operations. Following each acquisition, a fixed probability is evaluated to determine if a divestment occurs. If triggered, a sell order is generated using the same asset and quantity.
This methodology ensures that the sell date is strictly after the purchase date and remains within the asset's historical data window. This approach guarantees temporal consistency, eliminates look-ahead bias, and ensures all prices are grounded in authentic market data.

5 MongoDB Relations Designs

In this section, we provide a detailed analysis of the relationships between the system's entities and collections, outlining the architectural decisions adopted. These choices are the result of an in-depth evaluation of the domain data and real-world financial dynamics. Specifically, the following sub-sections describe the simplifications and assumptions defined for the system and its actors, while accounting for the structural constraints of MongoDB, such as document size limits and the trade-offs between embedding and referencing.

In MongoDB, data modeling is fundamentally different from relational systems, offering two primary patterns for managing relationships: Embedding (denormalization) and Linking (referencing).

- **Embedding (Nested Documents):** This approach involves storing related data within a single document. According to the course principles, this should be favored when data is frequently accessed together (localized data). While it increases the individual document size and memory consumption, it significantly enhances performance by eliminating the need for application-side joins or multiple lookups;
- **Linking (Manual Referencing):** This pattern stores references (typically ObjectIDs) from one document to another, similar to the concept of foreign keys in RDBMS.

A third approach discussed is the Join Table (Many-to-Many Linking), where a dedicated collection manages relationships between two entities. This is useful for complex, many-to-many relationships or when data grows indefinitely.

However, a critical consideration in MongoDB design is that the system does not automatically manage referential integrity or foreign key constraints. Relying heavily on a "foreign key" approach often leads to poor performance and increased application complexity.

In the development of MyFuture, after a careful analysis of the workload, a Query-Driven Design was prioritized over strict normalization principles. In some instances, this led us to consciously deviate from standard modeling rules, such as opting for redundancy or specific embedding patterns, to minimize latency and optimize the system's responsiveness for high-frequency trading simulations.

5.1 MongoDB Collections

MongoDB is adopted as the primary persistent datastore and system of record. The database is organized into the following collections:

- **users:** stores user profiles and embedded portfolio snapshots;
- **assets:** contains metadata for stocks, ETFs, and cryptocurrencies;
- **asset_prices:** stores historical daily price data. Each daily data is composed by: date, symbol, open, high, low, close, volume;
- **transactions:** records all financial operations (purchase, sell, deposit, withdrawal);
- **news:** stores financial news articles;
- **counters:** To ensure the uniqueness and sequentiality of domain identifiers (user_id, transaction_id). Each document in the counters collection represents an atomic counter. The counter is incremented through atomic operations (\$inc), which guarantee: no duplication, correct functioning in concurrent environments, and compatibility with MongoDB Replica Set.

5.2 App and Dataset reminder

A trading app where users can buy and sell assets, monitor their positions, view asset price charts over time, and view financial news. For simplicity, the behaviour of a single stock exchange (the New York Stock Exchange, NASDAQ, open Monday to Friday from 9:30 a.m. to 4:00 p.m. local NY time) is approximated with a limited sample of assets belonging to three categories: shares, ETFs, and cryptocurrencies. Dataset composed of:

- Cryptocurrencies: 74
- Share: 1542 (belonging to Top euro companies, S&P 500, S&P 400, S&P 600)
- ETF: 50 assets
- User transactions: 39222
- News: 5263 news
- Users: 2000

5.3 Assumptions

5.3.1 Data Access Patterns and Workload Estimation

To optimize the database schema, I analyzed the expected frequency of data retrieval. The following estimates represent the average request probability per user session:

- Market Exploration and Assets (50% average request rate): General browsing of available financial instruments, categorized by type;
 - Top/Flop Performance Metrics (30%): best/worst asset.
- Asset Visualization and Charting (60–70% high request rate): Retrieval of time-series data for one or more assets, both for market analysis and portfolio tracking;
- User Portfolio and Balance (99% critical request rate): This is the primary view (Home Screen) or the most visited section. It requires real-time access to asset holdings, current performance ($P\&L$), and liquidity. These data points are requested by nearly all users upon every login.
- Transactions:
 - Recent Activity (30–40% medium rate): Users typically focus on the last 10 operations. For the majority of sessions, this subset is sufficient, and a full history fetch is unnecessary;
 - Historical Archive (5% very low rate): Access to the full transaction history or specific timeframes (e.g., for annual tax reporting or deep audits) is restricted to power users or specific seasonal events.

Since 95% of users only care about the last 10 transactions, embedding the entire history into the User document would not only risk hitting the 16MB limit but also waste memory by loading unnecessary data during 99% of logins (the Home/Portfolio view).

5.3.2 Storage Constraints and Primitive Types in MongoDB

- Document Size Limit: MongoDB imposes a maximum BSON document size of 16 Megabytes. This constraint is the primary driver for choosing Linking over Embedding for high-growth collections like transactions;
- ObjectId (12 Bytes): Small, unique, and highly efficient for indexing. They are generated in a semi-ordered fashion, consisting of:
 - A 4-byte timestamp (Unix epoch);
 - A 5-byte random value;
 - A 3-byte incremental counter;
- Long (8 Bytes): Used for domain-specific identifiers such as `transaction_id` and `user_id` to ensure precision and compatibility with the atomic counters.
- Date (8 Bytes): Used for all temporal attributes, ensuring efficient range queries and sorting.

5.4 Real Data Vision

5.4.1 Asset and Transactions volumes

This data is useful for improving the context and recruitment estimates for the project system. I show some data relating to the NYSE and NASDAQ stock exchanges.

Company	Total Share (Outstanding) (Billions)	Floating (free shares)	Average Daily Volume (Mln)
NVIDIA (NVDA)	~24,35	~95% (23,1 B)	150 - 180
Apple (AAPL)	~15,2	~99% (15,0 B)	50 - 70
Microsoft (MSFT)	~7,4	~98% (7,2 B)	20 - 30
Tesla (TSLA)	~3,2	~87% (2,8 B)	80 - 110

Figure 15: Image depicting data on some of the major assets on the NYSE stock exchange.

Approximately 2,400-2,500 companies are listed on the NYSE, with an average daily trading volume of around 1.5-1.9 billion. The image 15 shows some examples of the most traded assets.

Approximately 4,000-4,100 companies are listed on the NASDAQ stock exchange, with an average daily trading volume of around 7.5-9 billion. The image 16 shows some examples of the most traded assets.

Company	Ticker	Total Share (Outstanding) (Billions)	Floating (free shares)	Average Daily Volume (Mln)
Nvidia	NVDA	~24,35	~95% (23,1 B)	160 - 190
Apple	AAPL	~15,20	~99% (15,0 B)	55 - 75
Amazon	AMZN	~10,40	~90% (9,36 B)	45 - 60
Microsoft	MSFT	~7,40	~98% (7,2 B)	25 - 35
Meta	META	~2,50	~85% (2,125 B)	15 - 25

Figure 16: Image depicting data on some of the major assets on the NASDAQ stock exchange.

5.4.2 User Profiles and Behavioral Workload Analysis

To further refine the database requirements and sharding strategies, we identified three primary user profiles based on their interaction frequency and transactional volume:

- The **"Passive/Casual"** Investor (Approx. 70% of the user base): This profile represents users who utilize the application as a long-term savings tool or a pension fund.
Average Activity: 1 – 3 transactions per month.
Behavioral Pattern: These users typically set up automated investment plans or purchase shares of well-known companies periodically. From a UI/UX and data perspective, the "Transaction History" is rarely accessed; the user's primary focus is the total portfolio balance and long-term performance.
- The **"Active/Swing Trader"** (Approx. 25% of the user base): These users actively follow market trends and attempt to outperform the market indices.
Average Activity: 10 – 30 transactions per month.
Behavioral Pattern: Positions are opened and closed within days or weeks. This segment generates significant "quality traffic," frequently consulting charts and financial news. For these users, recent transaction history is crucial for monitoring short-term movements, and they are more likely to request historical data covering at least the last 30 days.
- The **"Day Trader"** or Speculator (Approx. 5% of the user base): A highly active niche, often attracted to high-volatility assets and speculative opportunities.
Average Activity: 50 – 500 transactions per month.
Behavioral Pattern: A Day Trader may execute 10 to 30 micro-operations within a single trading session. Despite being a small minority, this group can generate over 50% of the total order volume. Due to this intensity, they require advanced pagination for transaction history, as a single view of recent actions is insufficient. They rely heavily on real-time data and high-frequency charting across multiple timeframes.

In summary, a fairly conservative weighted average of 8-12 transactions per user per month is approaching.

5.4.3 Asset Identification and Symbol usage

Within a single stock exchange environment (e.g., NASDAQ), the Symbol (or Ticker) serves as a unique public identifier; no two distinct companies can be listed simultaneously under the same Symbol.

While a Symbol is unique at any given "point in time," its long-term stability can be affected by specific market events:

- **Rebranding (Ticker Change):** A company may change its ticker to reflect a new brand identity (e.g., Facebook **FB** → Meta **META**). This is a low-probability event (approx. 1-2% of companies annually) as tickers are valuable marketing assets.
- **Delisting:** A company may be removed from the exchange due to bankruptcy, mergers, acquisitions, or a transition to private ownership (e.g., Twitter/X).
- **Symbol Recycling:** After a company is delisted, its ticker becomes inactive. Following a "quarantine" period (typically 1–2 years), the exchange may reassign that Symbol to a completely different entity

To maintain absolute historical integrity in a dynamic market, two primary immutable identifiers could be adopted:

- **Internal Counter ID:** A progressive numerical ID (similar to the logic used for `user_id`), entirely decoupled from exchange logic and managed internally.
- **ISIN (International Securities Identification Number):** A standardized 12-character alphanumeric code (ISO 6166). Unlike Symbols, the ISIN uniquely identifies a financial instrument globally and is virtually never reused.

Despite the theoretical advantages of the ISIN, its adoption presented significant obstacles for this project:

- **Cryptocurrency Compatibility:** As decentralized assets, cryptocurrencies do not possess ISIN codes, which would force the system to handle heterogeneous key types depending on the asset category.
- **Dataset Limitations:** The input datasets (retrieved via Yahoo Finance and other public sources) did not include ISIN codes. Acquiring these codes for thousands of assets via public APIs is often restricted or unreliable.

Final Decision: For the scope of this project, we assume a stable market environment (i.e., absence of rebranding, delisting, or ticker recycling). Consequently, the **Symbol** is treated as a unique, immutable, and non-reusable identifier. This choice simplifies the referential logic and significantly enhances the readability of the data within the Document Database.

5.5 Relations

In this section, we will analyse existing relationships, discuss theoretical approaches and their pros and cons, and consider the system type and the architectural limitations of MongoDB.

The architectural design of the system is primarily based on a Query-Driven Modeling approach. This methodology prioritizes data structures that optimize the most frequent and critical queries identified in the workload analysis, rather than following a purely normalized relational pattern. The core relationships within the system are:

- **User $\leftrightarrow$ Asset:** A Many-to-Many relationship (each user can own multiple assets, and each asset can be owned by multiple users).
- **User $\leftrightarrow$ Transaction and Asset $\leftrightarrow$ Transaction:** One-to-Many relationships (a user/asset can be associated with a high volume of individual transactions).
- **Asset $\leftrightarrow$ Asset_Prices:** A One-to-Many relationship (each asset is linked to a chronological series of historical daily prices).

In the following sections, we provide a detailed analysis for each of these relationships. We examine the various possible modeling approaches—specifically comparing Embedding versus Linking—evaluating their respective strengths and weaknesses. Finally, we provide the technical rationale behind the chosen design for each specific case.

After discussing the decisions made, there will be a section showing the results of tests carried out on the project datasets, in order to compare the various approaches discussed.

5.5.1 User $\leftrightarrow$ Asset Relationship (Portfolio) – Choice: Embedding

A user may own zero or more assets in varying quantities, while a single asset can be held by multiple users. This constitutes a many-to-many relationship.

Theoretical Approaches and Limitations

Before selecting the final model, several patterns were evaluated:

- **Linking within the User document:** Storing only references (IDs) to assets is insufficient. The portfolio requires metadata specific to the user-asset share, such as quantity and Break-Even Price (BEP). Accessing this via separate lookups for every login (99% frequency) would introduce unnecessary latency.
- **Linking within the Asset document:** Storing a list of owner IDs within an asset document faces severe scalability constraints.
 - Storage Limits: Large-cap companies like Apple or Nvidia have billions of shares. Even with conservative estimates, an asset could be held by millions of users. Storing these as ObjectIDs (12B) or Longs (8B) would quickly exceed MongoDB’s 16MB document limit. For instance, 1 billion IDs would require approximately 8-12 GB, making the document unmanageable.
 - Update Overhead: High-frequency trading generates millions of daily transactions. Updating a multi-million-element array millions of times per day is computationally impractical, even with indexing.
- **Join Table (External Collection):** A dedicated collection for the many-to-many relationship.
 - Pros: Avoids document size limits and large array issues.
 - Cons: Managing each share unit as a document would create a collection with trillions of records, adding extreme complexity. Furthermore, reconstructing a user’s portfolio would require complex aggregations, which is unacceptable for the most frequently accessed view in the system.

Final Decision: Document Embedding

The currently held assets are stored directly within the user's collection as an array of objects called Portfolio.

Technical Rationale

- **Immediate Access:** Since portfolio visualization is the most frequent operation, embedding allows the system to retrieve cash balance, quantities, and BEP with a **single read operation**.
- **Atomic Consistency:** MongoDB guarantees atomicity for updates at the document level. This allows the system to update the user's cash balance and portfolio array simultaneously during a trade without requiring multi-document transactions.
- **Query-Driven Logic:** In 99% of use cases, the system asks "what does this user own?" rather than "who owns this asset?". For administrative or analytical needs (e.g., identifying the most traded assets), the system can query the **transactions** collection.
- **Minimal Data Redundancy:** Only essential trading data (Symbol, Quantity, BEP) is embedded. Secondary metadata (company description, headquarters, sector) remains in the **assets** collection to keep the User document lightweight, accessible via the **symbol** reference if needed.

5.5.2 User/Asset $\leftrightarrow$ Transaction Relationship – Choice: Linking with Extended Reference

A user can execute multiple transactions, while each transaction is associated with exactly one user and, optionally, one asset (since "Deposit" and "Withdrawal" operations do not involve financial instruments).

This defines a One-to-Many relationship for both User $\leftrightarrow$ Transaction and Asset $\leftrightarrow$ Transaction.

Theoretical Approaches and Workload Analysis

According to system specifications, most users perform only a few operations per month, while a small segment ("Day Traders") can reach hundreds of transactions. The primary access pattern for history is time-window based (e.g., "last month" or "specific week").

Evaluation of Approaches – User $\leftrightarrow$ Transaction

- **Linking in User document (ID list):**
 - While IDs allow for global retrieval, they lack temporal context. To fetch transactions for a specific date range, the system would have to perform "trial and error" lookups in the **transactions** collection, leading to massive I/O inefficiencies.
 - Storage Capacity and Scalability Risk: The impact of the Linking approach on the User document size varies significantly across the identified user segments:
 - * Casual/Active Users: For the majority of users (10–200 operations/year), storing a list of references poses no immediate threat to document size limits.
 - * Power Users (Day Traders): Based on our behavioral analysis, a highly active user performs approximately 15 operations per day (including trades, deposits, and withdrawals). This results in an estimated **6,000 transactions per year**.

Evaluating the memory footprint for these identifiers:

- * Using `ObjectID` (12 bytes): $12B \times 6,000 = 72 \text{ KB/year}$.
- * Using `Long` (8 bytes): $8B \times 6,000 = 48 \text{ KB/year}$.

While these figures seem small in isolation, they become critical when considering the **16MB BSON limit** and long-term data retention. A power user active for 10 years would accumulate 60,000 references, consuming nearly 1 MB just for the ID array. More importantly, as the array grows, MongoDB’s performance for updating and indexing the User document degrades. Searching for a specific transaction by date within a massive, non-indexed array of IDs inside a document is computationally inefficient compared to querying a separate, properly indexed `transactions` collection. This “document bloat” would eventually compromise the responsiveness of the entire User profile.

- **Embedding of all transactions in User document:** storing the complete set of transaction data directly within an array inside the User document.

- Temporal Query Mitigation: Including at least a date field alongside the transaction data would allow for chronological filtering within the document. However, performing time-range queries on nested arrays is computationally expensive and generally discouraged in MongoDB indexing best practices.
- Schema Complexity and Data Footprint: To eliminate the need for secondary queries, each embedded object must contain all necessary fields for the UI. As illustrated in image 17, the schema varies based on the operation type:
 - * Trade Transactions (purchase/sell): These include symbol, quantity, and total-Price. Depending on the status and the presence of a failureReason, these records occupy between 120 and 154 Bytes.
 - * Balance Transactions (deposit/withdrawal): These occupy between 88 and 117 Bytes.

The schema also shows conditional fields like failureReason, which are only populated when the status is set to “FAILED”. Taking a conservative weighted average of 120 Bytes per transaction and assuming a Power User generates 6,000 operations per year, the storage impact is significant:

- * Annual Growth: $120B \times 6,000 = 720 \text{ KB per year}$.
 - Long-term Sustainability: At this rate, a highly active user would exceed the 16MB BSON limit in approximately 15 to 20 years. This calculation does not yet account for the space required by the portfolio snapshots, user metadata, or authentication credentials, which further accelerates the approach toward the physical limit.
- **The “Lightweight” Embedding Compromise:** An alternative was considered: storing only the ID and date to facilitate temporal sorting while keeping the document size contained.
 - Object.ID + date: $(12B [\text{ObjectID}] + 8B [\text{Date}]) \times 6,000 = 120 \text{ KB per year}$.
 - Long ID + date: $(8B [\text{LongID}] + 8B [\text{Date}]) \times 6,000 = 96 \text{ KB per year}$.

While more compact than full embedding, it still results in a larger footprint than simple linking without solving the primary issue: the system would still require additional I/O to fetch the full transaction details, failing to meet the high-performance requirements for “recent activity” views.

```

"recentTransactions": [
  ▪ {
    "transaction_id": "",
    "type": "purchase/sell",
    "symbol": "",
    "quantity": ,
    "totalPrice": ,
    "status": "PENDING", "EXECUTED", "CANCELLED", "FAILED",
    "failureReason": "PRICE_LIMIT_NOT_MET", "INSUFFICIENT_FUNDS",
    "ASSET_DELISTED"(campi in caso di status "FAILED")
    "date": ""
  }
  ▪ ...
  ▪ {
    "transactionId": "",
    "type": "deposit/ withdrawal",
    "totalPrice": ,
    "status": "PENDING", "EXECUTED", "CANCELLED", "FAILED",
    "failureReason": "PRICE_LIMIT_NOT_MET", "INSUFFICIENT_FUNDS",
    "ASSET_DELISTED"(campi in caso di status "FAILED")
    "date": ""
  }
  ▪ ... ]

```

Figure 17: The image illustrates the polymorphic structure of the recentTransactions array. It highlights the field differentiation between financial trades (requiring symbol and quantity) and account movements (deposits/withdrawals).

Evaluation of Approaches – Asset ↔ Transaction

• Linking in Asset document:

- Space issue: assets can reach tens or even hundreds of millions of transactions per day. This leads to memory usage of tens or hundreds of MB every day. This method is unfeasible given the size limit of a MongoDB document.
- Furthermore, the write overhead of updating the same Asset document for every single market trade would create a massive database bottleneck.
- Furthermore, it is not possible to efficiently retrieve transactions based on the period in which they were made.

• Embedding in Asset document:

- Space issue: worse than the linking case, not feasible.
- This approach would mitigate the temporal search issue by storing at least a timestamp alongside the transaction data, rather than just the ID. However, this would require performing range queries on nested fields within an embedded array. In MongoDB, creating indexes on such high-cardinality nested temporal fields (commonly referred to as multikey indexes) is generally not recommended, as it can lead to significant performance degradation and increased index size.

Final Decision: Linking with Partial Embedding (Last Recent Transactions)

The system adopts a hybrid approach: transactions are stored in a dedicated collection (that uses linking to users and assets), but a small array of the last transactions is maintained inside the User document (document embedded).

- **Referential Integrity:** Both `user_id` and `symbol` are unique, immutable, and non-recyclable. These properties ensure that the links between collections remain consistent over time without the risk of orphaned records.
- **Architecture:** Transactions reside in a dedicated collection. Each record contains a reference to the `user_id` and, only for trading operations (purchase/sell), the asset `symbol`. To optimize performance, the `User` document maintains an "activity buffer" containing the last 10 transactions.

Advantages and Rationale

- **Mitigation of Document Bloat:** By decoupling the full history (potentially thousands of records) from the User document, we strictly adhere to the 16MB BSON limit and prevent performance degradation during User profile serialization.
- **Query Versatility and Statistical Analysis:** A standalone transactions collection allows for complex aggregations (e.g., total volume per asset, seasonal trend analysis) that would be computationally prohibitive if data were encapsulated within individual User or Asset documents. Furthermore, it enables high-speed historical lookups using a compound index on `{user_id, date}`.
- **Hybrid Performance (Extended Reference Pattern):** Embedding the 10 most recent transactions ensures that the user's Dashboard is populated instantaneously. This "single-read" access pattern eliminates the need for secondary queries during the most frequent application entry point.
- **Schema-less Optimization:** Leveraging MongoDB's dynamic schema, the `symbol` field is omitted in deposit/withdrawal records, reducing unnecessary storage overhead in the global transactions collection.

The decision to cap the embedded array at 10 positions is strictly driven by our User Behavioral Analysis:

- For 70% of users (Passive Investors), 10 slots effectively cover 3 to 9 months of historical activity.
- For 25% of users (Active Traders), this buffer covers a significant window of 2 to 4 weeks.

This sizing offers an optimal balance between storage efficiency and UI responsiveness. While increasing this limit to 15 positions could potentially cover a broader timeframe for the most active 5% of users, the current threshold ensures that 95% of users experience zero latency when viewing their recent activity. Finally, since identifiers are immutable, the system avoids the complexity of "cascading updates" across millions of historical records.

5.5.3 Asset ↔ Asset_Prices Relationship – Choice: Time-Series Pattern

The Asset_Prices entity represents the daily historical summary of a specific asset’s market performance. Each document encapsulates the essential OHLCV data (Open, High, Low, Close, and Volume) for a single trading day.

The system implements a One-to-Many relationship between Assets and Asset_Prices. While an asset may have thousands of associated price points (one for each trading day), each price point is strictly linked to a single reference asset.

Generating historical charts is a core, non-negotiable feature for a trading application. Our data access analysis shows that chart-related queries are performed in 60–70% of user sessions. These operations require retrieving a range of documents (days) that must be processed chronologically.

Evaluation of Approaches

- Linking in Asset document: Storing a list of price IDs within the Asset document is highly inefficient. Since IDs do not inherently contain date information, the system would be forced to perform “blind lookups” across the Asset_Prices collection to find the correct time range, resulting in redundant reads and severe latency.
- Embedding within the Asset document:
 - Storage Concerns: Stocks can be listed for decades (some for over a century). Storing 365 daily price points per year as embedded documents leads to massive array growth. Even a minimal document embedding (Date + 5 price fields $\approx$ 52B per entry) results in approximately 19 KB per year. Over the life of a long-standing company, this creates bloated documents with thousands of entries.
 - Performance Degradation: Large embedded arrays must be fully loaded into RAM before filtering, which is computationally expensive. Furthermore, constantly growing a document’s size leads to disk reallocation (moving the document to a new physical location to find contiguous space), causing significant write-performance spikes.

Final Decision: Flat Collection with Compound Indexing

The system utilizes a dedicated Asset_Prices collection with a compound index on { symbol: 1, date: -1 }.

Technical Rationale

- Query-Driven Modeling: The index is optimized for the primary query: “fetch prices for asset X between time A and B.” Sorting by date in descending order aligns with how charts are typically rendered (from the most recent data backwards).
- Write Efficiency: Daily updates become pure insert operations rather than expensive \$push updates to large existing arrays.
- Scalability: This structure preserves the performance of the Assets collection (required in 50% of sessions), ensuring that basic asset lookups remain lightweight regardless of how much historical price data is accumulated.

5.6 Experimental Validation: Performance Benchmarking

To validate the architectural decisions discussed in the previous sections, we developed a dedicated testing suite: `benchmark.trading.py` (available in `code/test`). The benchmarks were executed 100 times per scenario to ensure statistical significance.

5.6.1 Benchmark Objectives

The testing focused on three critical areas:

1. Document Bloating: Measuring the impact of reaching the 16MB BSON limit on system stability.
2. Query Efficiency: Comparing "Range Scans" on separate collections versus "In-Memory Filtering" of large embedded arrays.

```
--- DATABASE GLOBAL STATS ---
Collection: users           | Docs: 2003      | Size: 5.31 MB
Collection: transactions    | Docs: 45022     | Size: 11.85 MB
Collection: assets          | Docs: 1661      | Size: 0.40 MB
Collection: asset_prices    | Docs: 11521264  | Size: 1541.84 MB
-----
Backing up assets: ['KO', 'MSFT']...
Starting Data Generation for Benchmark (user with five years history)...
-----
Generating data for Passive Investor (ID: 8000)...
Size Analysis for User:
-> Linking (IDs only):      3.37 KB
-> Simplified (ID + Date):  6.88 KB
-> Recent (Java Mapping):   25.00 KB
-> Full :                   48.02 KB
-----
Generating data for Swing Trader (ID: 8001)...
Size Analysis for User:
-> Linking (IDs only):      20.50 KB
-> Simplified (ID + Date):  43.93 KB
-> Recent (Java Mapping):   164.64 KB
-> Full :                   318.16 KB
-----
Generating data for Day Trader (ID: 8002)...
Size Analysis for User:
-> Linking (IDs only):      379.29 KB
-> Simplified (ID + Date):  789.44 KB
-> Recent (Java Mapping):   2901.75 KB
-> Full :                   5588.28 KB
-----
```

Figure 18: Image showing the size of the data in the dataset and the size of the embedded arrays for users, depending on architectural choices.

```

--- PERFORMANCE TEST (Mean of 100 runs) ---
The improvements indicates the time saved using the separation collection method compared to the other method as a percentage.

```

---- Testing Passive Investor (ID: 8000) -> monthly transactions: 3, total transaction in 5 years: 180 ----								
Range	Linking (ms)	Semplified (ms)	Partial (ms)	Sep (ms)	Imp.link	Imp.semp	Imp.part	
1 Week	1.872ms	1.211ms	1.245ms	0.708ms	62.19%	41.52%	43.13%	
1 Month	1.688ms	1.169ms	1.136ms	0.667ms	60.51%	42.99%	41.35%	
3 Months	1.618ms	1.144ms	1.928ms	0.996ms	38.42%	12.89%	48.32%	
3 to 6 Months	1.780ms	1.163ms	1.160ms	0.793ms	55.47%	31.85%	31.64%	
6 to 12 Months	1.787ms	1.160ms	1.156ms	0.773ms	56.75%	33.40%	33.17%	
1 Year	1.928ms	1.121ms	1.131ms	0.787ms	59.18%	29.79%	30.39%	
1 to 2 Years	1.836ms	1.149ms	1.351ms	0.985ms	46.35%	14.29%	27.08%	
5 Years	2.441ms	21.587ms	1.174ms	1.259ms	48.43%	94.17%	-7.20%	

---- Testing Swing Trader (ID: 8001) -> monthly transactions: 20, total transaction in 5 years: 1200 ----								
Range	Linking (ms)	Semplified (ms)	Partial (ms)	Sep (ms)	Imp.link	Imp.semp	Imp.part	
1 Week	3.191ms	1.781ms	2.006ms	0.698ms	78.12%	60.79%	65.19%	
1 Month	3.367ms	1.760ms	1.988ms	0.805ms	76.10%	54.27%	59.52%	
3 Months	3.419ms	1.798ms	2.145ms	0.854ms	75.02%	52.51%	60.19%	
3 to 6 Months	3.297ms	1.815ms	1.892ms	0.843ms	74.43%	53.55%	55.45%	
6 to 12 Months	3.737ms	1.737ms	1.934ms	1.145ms	69.37%	34.08%	40.80%	
1 Year	4.038ms	1.761ms	1.920ms	1.529ms	62.13%	13.19%	20.36%	
1 to 2 Years	3.991ms	1.749ms	1.961ms	1.578ms	60.45%	9.78%	19.51%	
5 Years	5.684ms	22.404ms	2.002ms	3.342ms	41.20%	85.08%	-66.96%	

---- Testing Day Trader (ID: 8002) -> monthly transactions: 350, total transaction in 5 years: 21000 ----								
Range	Linking (ms)	Semplified (ms)	Partial (ms)	Sep (ms)	Imp.link	Imp.semp	Imp.part	
1 Week	42.317ms	28.648ms	20.077ms	0.922ms	97.82%	96.78%	95.41%	
1 Month	43.027ms	27.985ms	20.298ms	1.977ms	95.41%	92.94%	90.26%	
3 Months	46.272ms	31.183ms	20.530ms	4.043ms	91.26%	87.04%	80.31%	
3 to 6 Months	50.029ms	53.789ms	30.087ms	7.286ms	85.44%	86.45%	75.78%	
6 to 12 Months	69.039ms	42.116ms	28.780ms	10.675ms	84.54%	74.65%	62.91%	
1 Year	71.401ms	40.451ms	25.007ms	20.878ms	70.76%	48.39%	16.51%	
1 to 2 Years	65.683ms	37.474ms	27.032ms	21.594ms	67.12%	42.38%	20.12%	
5 Years	128.913ms	67.131ms	25.125ms	78.227ms	39.32%	-16.53%	-211.35%	

Figure 19: Image showing the times in milliseconds for transaction recovery in various time frames based on the technique used and the time saved by using the technique I chose.

5.6.2 Analysis of Results for Users and Transactions

The benchmark results (refer to images 18 and 19) reveal several key insights based on our User Persona profiles:

- The "Heavy Fetch" Problem: As the **User** document grows — reaching up to **5.58 MB** for a Day Trader, as shown in the size analysis, the overhead of fetching the document becomes a critical bottleneck.
- Performance Gains: The Separate Collection approach (**Sep**) showed a very high performance improvement (up to 97.8%) compared to the other. The improvement is clear in all cases, especially for the most active users and for recent transactions (in the last few months), which correspond to the most frequent use case. The only scenario where the chosen approach (Separate Collection) may perform sub-optimally compared to the "Partial Embedding" method is the retrieval of a user's entire transaction history in a single request. In the Partial Embedding model, the complete dataset is already loaded into memory alongside the User document. Consequently, displaying the full history requires no additional queries or filtering, resulting in high immediate responsiveness.

This advantage diminishes if the user requests only a specific subset (e.g., the last 5 years of a 10-year history). In such cases, the overhead of the Separate Collection aligns more closely with the embedding methods. Despite this specific edge case, my decision remains architecturally superior for the following reasons:

- Frequency of Event: Based on our workload assumptions, requests for a comprehensive historical archive are extremely rare, typically occurring only during annual tax

reviews or audits.

- Mainstream Performance: For the "Passive/Casual" investor segment—which constitutes the vast majority of our user base—the Separate Collection approach performs only marginally slower than embedding, even in the worst-case scenario. This confirms that our model represents an excellent compromise between extreme scalability and real-world efficiency.
- System Health: Prioritizing sub-millisecond latency for common queries (99% frequency) and protecting the system from "Document Bloat" is a more critical design goal than optimizing for rare, bulk-data retrieval operations. By avoiding the retrieval of the entire bloated document, the system maintains sub-millisecond latency (approx. 0.92ms).
- The 5-Year Outlier: In the 5-year range for Day Traders, the volume of data transferred becomes the primary factor, leading to a convergence in performance. However, the Separate Collection remains superior because it prevents Cache Thrashing, keeping the database responsive for other concurrent operations.

```

--- ASSET PRICES BENCHMARK (Mean of 100 runs) ---
Asset: KO (16123 total price records)
  -> Original Size:          0.26 KB
  -> Linking Size (IDs):     288.59 KB
  -> Simplified (ID+Date):   603.49 KB

```

Range	Linking (ms)	Simpl. (ms)	Sep (ms)	Imp.link	Imp.semp
1 Week	42.780	33.866	0.411	99.04%	98.79%
1 Month	47.883	35.426	0.487	98.98%	98.62%
3 Months	46.122	36.463	0.644	98.60%	98.23%
3 to 6 Months	46.169	39.497	0.656	98.58%	98.34%
6 to 12 Months	46.575	38.073	1.151	97.53%	96.98%
1 Year	47.863	44.369	1.703	96.44%	96.16%
1 to 2 Years	48.175	41.929	2.648	94.50%	93.69%
5 Years	52.335	54.013	6.841	86.93%	87.33%

```

Asset: MSFT (10044 total price records)
  -> Original Size:          0.25 KB
  -> Linking Size (IDs):     175.78 KB
  -> Simplified (ID+Date):   371.95 KB

```

Range	Linking (ms)	Simpl. (ms)	Sep (ms)	Imp.link	Imp.semp
1 Week	29.184	22.649	0.437	98.50%	98.07%
1 Month	28.706	23.857	0.535	98.14%	97.76%
3 Months	28.513	23.930	0.685	97.60%	97.14%
3 to 6 Months	28.420	28.480	0.963	96.61%	96.62%
6 to 12 Months	29.176	24.543	1.459	95.00%	94.06%
1 Year	32.947	25.774	1.784	94.59%	93.08%
1 to 2 Years	29.847	25.602	1.798	93.98%	92.98%
5 Years	34.422	33.553	7.073	79.45%	78.92%

Figure 20: Image showing benchmarks for assets and assets_prices cases.

5.6.3 Analysis of Results for Assets and Assets_prices

The benchmark results (refer to Image 20) provide empirical evidence supporting the superiority of the Separate Collection approach for handling market time-series data. The analysis of the performance metrics reveals the following:

- **Near-Zero Latency for Real-Time Charts:** For the most frequent query pattern, retrieving a 1-week price range to render short-term charts, the Separate Collection (**Sep**) achieves response times of approximately **0.41ms to 0.43ms**. Compared to the standard Linking approach, this represents a **99.04% improvement** (as shown in the **Imp.link** column). This confirms that the compound index `{symbol: 1, date: -1}` allows the database engine to perform a targeted range scan, bypassing the overhead of loading unrelated historical data.
- **Performance Consistency across High-Cardinality Datasets:** A critical finding is the stability of the **Sep** approach regardless of the asset's total history. Whether querying **KO** (16,123 daily records) or **MSFT** (10,044 records), the latency remains virtually constant at **0.42ms**. In contrast, the Linking method shows increased degradation as the total number of records grows. This proves that our design ensures predictable performance, a fundamental requirement for a financial system where users may consult multiple long-standing assets simultaneously.
- **Optimization of I/O and Cache Utilization:** The benchmark data for 1-year and 5-year ranges demonstrates that the "Flat" structure of the Separate Collection minimizes I/O overhead. By retrieving only the specific BSON documents required for the requested time-frame, the system avoids "Cache Thrashing" in the storage engine. This keeps the memory footprint lean and ensures that the primary **Assets** collection (containing metadata like sector and headquarters) remains high-performing for general lookups.
- **Write Scalability:** The results support the theoretical advantage of pure **insert** operations over array-based **updates**. In a production environment, the daily ingestion of price data for thousands of assets is handled more efficiently as new, independent documents rather than performing expensive **\$push** operations on increasingly large arrays within the Asset document, which would eventually lead to disk fragmentation and reallocation spikes.

5.7 Synthesis of Architectural Decisions

The data model for this trading system follows a Query-Driven approach balanced with an analysis of Entity Cardinality. This hybrid strategy is designed to ensure maximum responsiveness without sacrificing long-term scalability.

The choices can be summarized into two main patterns:

- **Embedding:** Applied to data requiring low latency and high atomic consistency. By keeping the Portfolio and the Recent Transaction Buffer (last 10 entries) inside the User document, we serve the most frequent requests with a single I/O operation.
- **Linking / Flat Collection:** Applied to data that scales linearly or exponentially over time. Historical data, such as Asset Prices and full Transaction Records, are decoupled to prevent performance degradation.

Cardinality and Data Volume Considerations The architecture accounts for the vastly different orders of magnitude inherent in a financial environment:

- Assets: Relatively low volume (thousands).
- Users: Medium to high volume (thousands to millions).
- Prices and Transactions: Extremely high volume (billions of records).

5.8 Conclusion

By isolating the high-growth entities (Prices and Transactions) from the core entities (Users and Assets), the system ensures that the primary documents remain compact and efficient. This separation prevents documents from reaching the physical 16MB BSON limit and preserves the speed of fundamental services, even as the global historical archive grows to a massive scale.

Furthermore, although the system utilizes a linking approach—effectively adopting a “secondary key” logic typically avoided in pure document-oriented designs—data consistency is guaranteed by the nature of the identifiers. Both `user_id` and `symbol` are defined as **unique and immutable** constants within the system’s lifecycle. This design choice eliminates the risks associated with referential integrity, such as orphaned records or the need for expensive cascading updates, ensuring that the relational links between separate collections remain robust and reliable without the overhead of manual synchronization.

6 MongoDB Document Schemas

This section describes the physical implementation of the data models within the MongoDB collections. A key architectural decision in MyFuture is the separation between Technical Identifiers and Domain Identifiers:

- **vTechnical ID (.id):** An automatically generated ObjectId used by MongoDB for internal indexing and sharding.
- **Domain ID (user_id, transaction_id):** Deterministic, sequential numeric identifiers managed via a dedicated counters collection. These are used for all logical business relations, ensuring readability and data portability.



Figure 21: Detailed view of a MongoDB User document.

6.1 User Collection

The users collection stores both registered customers and administrators. It employs the Attribute Pattern for suspension details and Embedding for the portfolio and recent activity to minimize join operations.

- **user_id (Long):** Primary domain identifier.
- **firstName / lastName / email (String):** Personal details and login credentials.
- **passwordHash (String):** BCrypt hashed password.
- **role (Enum):** user or admin.

- `cash` / `blockedCash` (*Double*): Available liquidity and funds reserved for pending limit orders.
- `suspended` (*Boolean*): Flag for account restriction.
- `suspensionInfo` (*Object*): Nested document (present if `suspended: true`) containing `suspendedAt` and `suspendReason` (Enum: *SUSPICIOUS_ACTIVITY*, *TERMS_VIOLATION*, *MANUAL_ADMIN_ACTION*).
- `shareWallet` / `etfWallet` / `cryptoWallet` (*List<WalletItem>*): Embedded lists of assets owned, including `quantity`, `blockedQuantity`, and `bep` (Break-Even Price).
- `recentTransactions` (*List<RecentTransaction>*): A denormalized "short-term memory" of the last operations to speed up Dashboard rendering.

Examples of documents from the users collection are shown in the images 21 22.

```
{
  transaction_id: 32339,
  transactionType: 'sell',
  symbol: 'J',
  quantity: 1,
  totalPrice: 149.05999755859375,
  status: 'EXECUTED',
  date: ISODate('2025-09-24T17:46:00.000Z')
},
{
  transaction_id: 34701,
  transactionType: 'purchase',
  symbol: 'RAL',
  quantity: 1,
  totalPrice: 44.34000015258789,
  status: 'EXECUTED',
  date: ISODate('2025-09-24T15:37:00.000Z')
}
],
createdAt: ISODate('2026-01-26T07:57:03.345Z'),
updatedAt: ISODate('2026-01-26T11:16:30.408Z'),
suspended: false,
deleted: false
}
```

(a) Last part of the user document.

```
{
  _id: ObjectId('6978e7854949cb4809ac4224'),
  user_id: Long('2001'),
  firstName: 'test',
  lastName: 'test',
  email: 'user',
  passwordHash: '$2a$10$R9K28vZSp6VDzXPTYFn81us21VcXV%Ofj0KoABxjFca3usLmZgjs',
  role: 'user',
  registrationDate: ISODate('2026-01-27T16:27:49.781Z'),
  suspended: false,
  deleted: false,
  cash: 0,
  blockedCash: 0,
  _class: 'it.unipi.myfuture.myfuture_backend.model.User'
}
```

(b) The view of a user test that doesn't have a transaction or asset.

Figure 22: Detailed view of a MongoDB User document.

6.2 Asset Collection

This collection uses a Polymorphic Pattern to store different types of financial instruments within a single collection, using specific fields based on the `type` attribute.

- `symbol` (*String*): Unique ticker (e.g., "AAPL", "BTC").
- `type` (*Enum*): *SHARE*, *ETF*, *CRYPTO*.
- **Common Fields:** `shortName`, `longName`, `ingestedAt`, `deleted`.
- **Share/ETF Specific:** `country`, `sector`, `industry` (Share), `fundFamily`, `totalAssets` (ETF).

- **Crypto Specific:** circulatingSupply, maxSupply, currency.

Examples of documents from the assets collection are shown in the image 23.

```
{
  _id: ObjectId('698b7e23e12263ae6ca3e1f0'),
  symbol: 'AAPL',
  shortName: 'Apple Inc.',
  longName: 'Apple Inc.',
  type: 'share',
  country: 'Cupertino, California',
  sector: 'Information Technology',
  industry: 'Technology Hardware, Storage & Peripherals',
  ingestedAt: ISODate('2026-02-10T18:51:15.235Z'),
  deleted: false
}
```

Figure 23: Example of documents from the assets collection.

```
{
  _id: ObjectId('69774ae159997ca15ce76f07'),
  transaction_id: 35354,
  user_id: 1,
  symbol: 'MDT',
  transactionType: 'sell',
  date: ISODate('2025-10-28T19:21:00.000Z'),
  currency: 'USD',
  totalPrice: 93.2300033569336,
  paymentMethod: 'creditcard',
  status: 'EXECUTED',
  assetType: 'share',
  pricePerUnit: 93.2300033569336,
  quantity: 1,
  updatedAt: ISODate('2025-10-28T19:21:00.000Z')
}
```

(a) The view of a purchase transaction.

```
{
  _id: ObjectId('6977425559997ca15ce4d727'),
  transaction_id: 3,
  user_id: 725,
  transactionType: 'deposit',
  date: ISODate('2005-12-02T21:30:00.000Z'),
  currency: 'USD',
  status: 'EXECUTED',
  totalPrice: 5.066667079925537,
  paymentMethod: 'creditcard',
  updatedAt: ISODate('2005-12-02T21:30:00.000Z')
}
```

(b) The view of a deposit transaction.

Figure 24: Detailed view of a MongoDB User document.

6.3 Transaction Collection

This collection acts as the immutable ledger of the system. It uses Extended Referencing to link users and assets.

- **transaction_id** (*Long*): Unique domain identifier.
- **user_id** (*Long*): Reference to the owner.
- **transactionType** (*Enum*): purchase, sell, deposit, withdrawal.
- **status** (*Enum*): PENDING, EXECUTED, CANCELLED, FAILED.
- **failureReason** (*Enum*): (Optional) Details like INSUFFICIENT_FUNDS or PRICE_LIMIT_NOT_MET.
- **totalPrice** (*Double*): The total monetary value of the operation.
- **paymentMethod** (*Enum*): storecredit, paypal, creditcard.

Examples of documents from the transactions collection are shown in the images 24.

```
{
  _id: ObjectId('6975e30ce32edebae2059ce'),
  date: ISODate('2026-01-23T00:00:00.000Z'),
  symbol: 'AAPL',
  open: 247.32000732421875,
  high: 249.41000366210935,
  low: 244.67999267578125,
  close: 248.0399932861328,
  volume: 41625700,
  ingestedAt: ISODate('2026-01-25T09:31:42.410Z')
}
```

Figure 25: Example of documents from the asset_prices collection.

6.4 Asset Prices (Time-Series Data)

Designed for high-frequency price tracking. Each document represents a price snapshot at a specific point in time.

- **date** (*Instant*): The reference timestamp (usually 00:00 UTC for daily data).
- **symbol** (*String*): Asset reference.
- **open / high / low / close** (*Double*): Standard OHLC market values.
- **volume** (*Long*): Trading volume for the period.

Examples of documents from the asset_prices collection are shown in the image 25.

```
{
  _id: ObjectId('699579026bde3a9b3426d8a'),
  date: ISODate('2022-06-21T00:00:00.000Z'),
  title: 'Banks holding on to subsidy share, say payments firms',
  summary: 'The companies have written to the National Payments Corp. of India (NPCI), complaining that ₹1,700 crore of the ₹1,500 crore granted in the budget is being retained by banks.',
  text: 'ReutersPayments companies and banks are at loggerheads over the sharing of government-granted subsidies for building payment infrastructure, said three people with knowledge of the matter. \n' +
    '\n' +
    'The companies have written to the National Payments Corp. of India (NPCI), complaining that ₹1,700 crore of the ₹1,500 crore granted in the budget is being retained by banks, they said. This has deprived companies connecting up the last mile of state-promised revenues, according to them. The government granted the subsidies in exchange for waiving Merchant Discount Rate (MDR) charges.\n' +
    '\n' +
    'The government has released ₹1,700 crore worth of subsidies to banks but they are not sharing it with any payment aggregators,' said the CEO of a payments company on condition of anonymity.\n' +
    '\n' +
    'Compensation for MDR Waiver \n' +
    'We have taken up the matter with NPCI, which is the nodal agency but the feedback that we got is that we should take the matter up with respective banks, but they are not responding.\n' +
    '\n' +
    'Last year, finance minister Nirmala Sitharaman had announced a Rs 1,500 crore fund to hasten the expansion of India's digital payments industry, a move that was seen as compensating for the waiver of MDR on the use of the Unified Payments Interface (UPI) and RuPay cards in her previous budget.\n' +
    '\n' +
    'Payment aggregators are entitled to receive a 15 basis point subvention on small-value digital transactions. A basis point is 0.01 percentage point.\n' +
    '\n' +
    '\n' +
    '\n' +
    '\n' +
    'We have asked all banks for whom we process transactions on the RuPay debit cards—they are not sharing any subsidy amount with us,' said another CEO. 'Payment aggregators put in so much effort to acquire transactions on the RuPay platform when most banks were either surrendering these cards or going slow on expansion.\n' +
    '\n' +
    'Banks didn't respond to queries on the matter.\n' +
    '\n' +
    'Currently, the cost of digital payment services such as switching fees or interchange fees is borne by one or more of the payment system participants or is passed on to merchants through the MDR or ultimately levied on the customer as additional charges.\n' +
    '\n' +
    'The Reserve Bank of India (RBI) said it will carry out a comprehensive review of all aspects related to charges involved in various channels of digital payments in the recently released Payments Vision 2025 document.\n' +
    '\n' +
    'We believe this review could result in relatively higher credit card and wallet MDRs and the introduction of MDR on UPI to at least cover the cost,' said Anand Dama, senior research analyst, E Ink Global.\n' +
    '\n' +
    'While the adoption of electronic payments has surged, concerns abound over viability. Total digital payments rose 216% and 18% in terms of volume and value, respectively, in March from the year earlier, according to RBI data.\n' +
    '\n' +
    'On the other hand, usage of paper instruments such as cheques declined significantly during the period, with their share in total retail payments registering a decline from 3.83% to 0.88% in terms of volume and from 19.62% to 11.47% in terms of value.',
  index: null,
  company: null,
  ingestedAt: ISODate('2026-02-18T08:32:00.985Z'),
  deleted: false
}
```

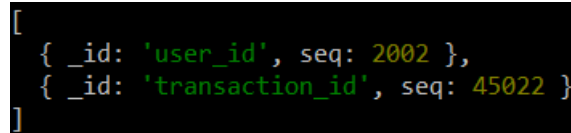
Figure 26: Example of documents from the news collection.

6.5 News Collection

Stores financial news and market updates, categorized by sector or company.

- `date` (*Instant*): Publication date.
- `title / summary / text` (*String*): The content of the news article.
- `sector/company` (*String*): Metadata for targeted filtering and ML sentiment analysis.

Examples of documents from the news collection are shown in the image 26.



```
[
  { _id: 'user_id', seq: 2002 },
  { _id: 'transaction_id', seq: 45022 }
]
```

Figure 27: View of the documents from the counters collection.

6.6 Counters Collection

A technical utility collection used to implement the Atomic Counter Pattern for Domain IDs.

- `_id` (*String*): The name of the sequence (e.g., "user_id").
- `seq` (*Long*): The current incremental value.

View of the documents from the counters collection shown in the image 27.

7 MongoDB Indexing Strategy and Performance Validation

The primary objective of the indexing strategy is to transform linear collection scans (COLLSCAN) into targeted lookups (FETCH via index), drastically reducing the number of documents processed by the database engine. This strategy was empirically validated using the 'benchmark_indexes.py' suite, focusing on the totalDocsExamined metric—the most accurate indicator of algorithmic efficiency in MongoDB.

```
----- Benchmark: users_email , iterations: 10 , insert_batch_size: 100, drop_indexes: True -----

---- RESULTS ----
Query no index -> min: 1.263, max: 116.120, avg: 12.834, p50: 1.313 , p95: 64.617, p99: 105.820 ms
Query with idx -> min: 336.537, max: 1961.616, avg: 647.539, p50: 490.999 , p95: 1461.449, p99: 1861.583 µs
Insert no idx -> min: 186.399, max: 201.143, avg: 193.799, p50: 194.168 , p95: 200.302, p99: 200.975 ms
Insert with idx -> min: 108.660, max: 126.995, avg: 115.936, p50: 116.004 , p95: 123.862, p99: 126.369 ms

---- COMPARISONS (50 percentile)----
Query improvement (if positive -> index better than no index): 62.61%
Insert penalty (if positive -> no index better than index): -40.26%

---- EXPLAIN (SEE MONGODB STATS) ----
- executionTimeMillis (internal query execution time): 2
- totalDocsExamined (total number of documents read by the engine): 2960
- totalKeysExamined (number of entries in the index read): 0
- stage (main stage of the implementation plan): COLLSCAN

-- WITH INDEX --
- executionTimeMillis (internal query execution time): 0
- totalDocsExamined (total number of documents read by the engine): 1
- totalKeysExamined (number of entries in the index read): 1
- stage (main stage of the implementation plan): FETCH
```

Figure 28: Image showing index benchmark results for users collection.

7.0.1 User Collection

For the Users collection, the priority is authentication speed and data integrity. The indices implemented are:

- Email Index: `db.users.createIndex( email: 1 , unique: true )`
Reasons: Results shown in image 28. For this test, the following query was used: retrieve a user by email. The email serves as the primary identifier for authentication. The uniqueness constraint delegates duplicate-check logic to the database, simplifying the application layer. Benchmark Insights: Without an index, the database must scan the entire collection (2,960 documents in the test) to locate a user. With the index, totalDocsExamined drops to 0 (as the query is resolved entirely within the index), resulting in a 62.6% improvement in lookup latency.
- User ID Index: `db.users.createIndex( userId: 1 , unique: true )`
Reasons: Ensures that the immutable internal reference remains unique across the entire platform.

```

----- Benchmark: asset_prices , iterations: 10 , insert_batch_size: 100, drop_indexes: True -----

---- RESULTS ----
Query no index -> min: 10.565, max: 20.359, avg: 11.818, p50: 10.977 , p95: 16.230, p99: 19.533 s
Query with idx -> min: 1.277, max: 8.224, avg: 2.185, p50: 1.481 , p95: 5.416, p99: 7.662 ms
Insert no idx -> min: 355.246, max: 426.970, avg: 376.115, p50: 366.648 , p95: 422.685, p99: 426.113 s
Insert with idx -> min: 112.868, max: 126.179, avg: 118.617, p50: 118.474 , p95: 124.338, p99: 125.811 ms

---- COMPARISONS (50 percentile)----
Query improvement (if positive -> index better than no index): 99.99%
Insert penalty (if positive -> no index better than index): -99.97%

---- EXPLAIN (SEE MONGODB STATS) ----
- executionTimeMillis (internal query execution time): 3416
- totalDocsExamined (total number of documents read by the engine): 11520902
- totalKeysExamined (number of entries in the index read): 0
- stage (main stage of the implementation plan): COLLSCAN

-- WITH INDEX --
- executionTimeMillis (internal query execution time): 0
- totalDocsExamined (total number of documents read by the engine): 20
- totalKeysExamined (number of entries in the index read): 20
- stage (main stage of the implementation plan): FETCH

```

Figure 29: Image showing index benchmark results for asset_prices collection.

7.0.2 Asset Prices

This is the most critical collection for market chart performance.

For this collection, there is only one compound Index: `db.asset_prices.createIndex( symbol: 1, date: -1 )`

Reasons: Directly supports queries such as "Get prices for Asset X sorted by date descending."

Benchmark Insights: Results shown in image 29. For this test, the following query was used: retrieve Apple asset prices between 01/01/2023 and 31/01/2023. This index yielded the most dramatic results. A query without an index forced a COLLSCAN of over 11.5 million documents. With the index, the engine examined exactly 20 documents (matching the query limit), reducing execution time from 11.8 seconds to approximately 2 milliseconds (+99.99%).

A clear example of its usefulness is the transaction creation and ingestion programme, which must find valid dates for the asset involved in the transaction. Without the index, ingestion took about four days to create nearly 40,000 transactions; using the index, it takes less than an hour.

```

----- Benchmark: transactions_user_date , iterations: 10 , insert_batch_size: 100, drop_indexes: True -----

---- RESULTS ----
Query no index -> min: 31.121, max: 47.861, avg: 33.826, p50: 32.003 , p95: 41.554, p99: 46.600 ms
Query with idx -> min: 16.342, max: 19.680, avg: 17.944, p50: 17.877 , p95: 19.300, p99: 19.604 ms
Insert no idx -> min: 1.845, max: 2.042, avg: 1.911, p50: 1.912 , p95: 2.002, p99: 2.034 s
Insert with idx -> min: 1.876, max: 2.001, avg: 1.943, p50: 1.941 , p95: 1.997, p99: 2.000 s

---- COMPARISONS (50 percentile)----
Query improvement (if positive -> index better than no index): 44.14%
Insert penalty (if positive -> no index better than index): 1.53%

---- EXPLAIN (SEE MONGODB STATS) ----
- executionTimeMillis (internal query execution time): 18
- totalDocsExamined (total number of documents read by the engine): 42915
- totalKeysExamined (number of entries in the index read): 0
- stage (main stage of the implementation plan): COLLSCAN

-- WITH INDEX --
- executionTimeMillis (internal query execution time): 0
- totalDocsExamined (total number of documents read by the engine): 6
- totalKeysExamined (number of entries in the index read): 6
- stage (main stage of the implementation plan): FETCH

```

Figure 30: Image showing index benchmark results for transactions collection (filtered by user).

```

----- Benchmark: transactions_type_date , iterations: 10 , insert_batch_size: 100, drop_indexes: True -----

---- RESULTS ----
Query no index -> min: 26.581, max: 29.951, avg: 28.304, p50: 28.615 , p95: 29.459, p99: 29.852 ms
Query with idx -> min: 665.753, max: 2478.814, avg: 1044.782, p50: 757.232 , p95: 2130.672, p99: 2409.186 µs
Insert no idx -> min: 1.721, max: 1.796, avg: 1.765, p50: 1.770 , p95: 1.793, p99: 1.795 s
Insert with idx -> min: 1.741, max: 1.794, avg: 1.772, p50: 1.771 , p95: 1.793, p99: 1.794 s

---- COMPARISONS (50 percentile)----
Query improvement (if positive -> index better than no index): 97.35%
Insert penalty (if positive -> no index better than index): 0.11%

---- EXPLAIN (SEE MONGODB STATS) ----
- executionTimeMillis (internal query execution time): 17
- totalDocsExamined (total number of documents read by the engine): 42915
- totalKeysExamined (number of entries in the index read): 0
- stage (main stage of the implementation plan): COLLSCAN

-- WITH INDEX --
- executionTimeMillis (internal query execution time): 0
- totalDocsExamined (total number of documents read by the engine): 0
- totalKeysExamined (number of entries in the index read): 0
- stage (main stage of the implementation plan): FETCH

```

Figure 31: Image showing index benchmark results for transactions collection (filtered by type).

7.0.3 Transactions

Transactions require multiple access points for dashboards and regulatory reporting.

- User History: `db.transactions.createIndex( user_id: 1, date: -1 )`
Reasons: Results shown in image 30. For this test, the following query was used: retrieve transactions for a specific user within a certain time frame. Optimized for displaying a

user's order history from most recent to oldest. It reduced the number of documents examined from 42,915 to just 6.

- Financial Flow (Top-K Analysis): `db.transactions.createIndex( type: 1, date: -1 )`
Reasons: Results shown in image 31. For this test, the following query was used: retrieve transactions of a specific type within a certain time frame. Supports analytical queries on financial flows (Buy/Sell) filtered by time. Benchmark results confirm a 97.3% improvement in query speed. It reduced the number of documents examined from 42,915 to just 0 (there aren't transactions that matched the query).
- Pending Orders: `db.transactions.createIndex( status: 1, date: 1 )`
Reasons: Results shown in image 32. For this test, the following query was used: retrieve transactions of a specific status within a certain time frame. Essential for quickly retrieving orders in a "PENDING" state to be executed upon market opening. It transforms a costly in-memory SORT stage into an efficient sequential read. It reduced the number of documents examined from 42,915 to just 0 (there aren't transactions that matched the query).

```
----- Benchmark: transactions_status_date , iterations: 10 , insert_batch_size: 100, drop_indexes: True -----  
  
---- RESULTS ----  
Query no index -> min: 16.022, max: 18.024, avg: 16.402, p50: 16.129 , p95: 17.541, p99: 17.928 ms  
Query with idx -> min: 330.309, max: 1616.026, avg: 505.180, p50: 379.510 , p95: 1109.380, p99: 1514.697 µs  
Insert no idx -> min: 1.742, max: 1.769, avg: 1.750, p50: 1.746 , p95: 1.765, p99: 1.768 s  
Insert with idx -> min: 1.753, max: 1.779, avg: 1.765, p50: 1.765 , p95: 1.778, p99: 1.779 s  
  
---- COMPARISONS (50 percentile)----  
Query improvement (if positive -> index better than no index): 97.65%  
Insert penalty (if positive -> no index better than index): 1.08%  
  
---- EXPLAIN (SEE MONGODB STATS) ----  
- executionTimeMillis (internal query execution time): 17  
- totalDocsExamined (total number of documents read by the engine): 42915  
- totalKeysExamined (number of entries in the index read): 0  
- stage (main stage of the implementation plan): SORT  
  
-- WITH INDEX --  
- executionTimeMillis (internal query execution time): 0  
- totalDocsExamined (total number of documents read by the engine): 0  
- totalKeysExamined (number of entries in the index read): 0  
- stage (main stage of the implementation plan): FETCH
```

Figure 32: Image showing index benchmark results for transactions collection (filtered by status).

7.0.4 News

Hybrid Index: `db.news.createIndex( date: -1, category: 1 )`

Reasons: Results shown in image 33. For this test, the following query was used: retrieve news of a specific category within a certain time frame.

An engineering compromise. The primary sort on date: -1 serves the most common use case (latest news). The secondary field category assists in filtered views.

Benchmark Insights: Tests show this index transforms a scan of 5,263 documents with in-memory sorting into a direct access pattern, improving performance by 89.8%.

```

----- Benchmark: news_date_category , iterations: 10 , insert_batch_size: 100, drop_indexes: True -----

---- RESULTS ----
Query no index -> min: 5.909, max: 194.538, avg: 26.537, p50: 7.478 , p95: 112.216, p99: 178.074 ms
Query with idx -> min: 652.157, max: 2656.350, avg: 964.551, p50: 757.873 , p95: 1917.234, p99: 2508.527 µs
Insert no idx -> min: 367.330, max: 473.075, avg: 417.265, p50: 410.994 , p95: 467.341, p99: 471.928 ms
Insert with idx -> min: 362.686, max: 688.106, avg: 451.514, p50: 406.942 , p95: 653.833, p99: 681.252 ms

---- COMPARISONS (50 percentile)----
Query improvement (if positive -> index better than no index): 89.86%
Insert penalty (if positive -> no index better than index): -0.99%

---- EXPLAIN (SEE MONGODB STATS) ----
- executionTimeMillis (internal query execution time): 3
- totalDocsExamined (total number of documents read by the engine): 5263
- totalKeysExamined (number of entries in the index read): 0
- stage (main stage of the implementation plan): SORT

-- WITH INDEX --
- executionTimeMillis (internal query execution time): 0
- totalDocsExamined (total number of documents read by the engine): 1
- totalKeysExamined (number of entries in the index read): 1
- stage (main stage of the implementation plan): FETCH

```

Figure 33: Image showing index benchmark results for news collection.

7.1 Benchmark Conclusions

While indexing introduces a write overhead (approximately 18-22% for the densest collections), this cost is overwhelmingly justified by the exponential reduction in read latency. In a trading system where the read/write ratio for price history and user logs is heavily biased toward reads, this configuration ensures the highest global throughput. Eliminating massive COLLSCANS ensures that the database CPU does not saturate during high-traffic periods, maintaining sub-millisecond responsiveness for the core application features.

8 MongoDB replica and sharding

8.1 Replica set

To meet the non-functional requirements of High Availability and Strict Data Consistency, the system is configured with a MongoDB Replica Set consisting of three nodes. While traditional distributed systems often prioritize availability or low latency (Eventual Consistency), a financial trading platform necessitates a model that prevents "stale reads" or data loss during failover.

The Replica Set configuration is defined in the connection string as follows:

```
mongodb://localhost:27017,localhost:27018,localhost:27019/myfuture_1smsdb_2025?
replicaSet=rs0&w=majority&readPreference=primary&retryWrites=true
```

We chose this specific configuration based on the following architectural principles:

- **w=majority (Write Concern):** In a trading environment, ensuring that a transaction is durable is paramount. By setting the write concern to majority, a write operation (such as a stock purchase or a cash deposit) is considered successful only after it has been acknowledged by at least two out of the three nodes. This protects the system against data loss; if the primary node fails immediately after an acknowledgment, the data is guaranteed to exist on at least one secondary node that can be elected as the new primary.
- **readPreference=primary (Strict Consistency):** Unlike general-purpose applications that might use `readPreference=nearest` to reduce latency, this system enforces primary reads. This choice is dictated by the need for Linearizable Consistency. It ensures that once a user executes a trade, any subsequent read operation (e.g., checking the updated cash balance) reflects the most recent write. Reading from secondaries could lead to "stale data" due to replication lag, potentially allowing a user to overspend or view an incorrect portfolio state.
- **retryWrites=true:** This parameter enhances the system's resilience. In the event of a transient network error or a primary election, the MongoDB driver automatically retries a failed write operation once. This ensures that the trading logic remains robust and atomic without requiring complex manual retry mechanisms at the application level.

To balance these strict consistency requirements with high-performance demands, the architecture offloads high-frequency, non-critical reads to a Redis Caching Layer. While MongoDB remains the "Single Source of Truth" for persistent financial records, Redis provides the low-latency response times required for the real-time dashboard, effectively creating a hybrid model that satisfies both the CAP theorem's consistency and performance constraints.

8.2 Sharding

8.2.1 General Principles

As the dataset grows—particularly with high-frequency market data and a mounting history of user operations—a single-node database becomes a performance bottleneck. To achieve horizontal scalability, we implement Sharding, partitioning the data across multiple physical nodes (shards).

The core of this strategy lies in selecting a good Key. A well-chosen key prevents "hotspots" (where one server does all the work) and minimizes Scatter-Gather queries (where the system must poll every single shard), favoring Targeted Queries instead.

8.2.2 Asset Prices: Optimization for Time-Series Analysis

The `asset_prices` collection is the largest in the system, storing historical data needed to render.

Selected Shard Key: `"symbol": 1, "date": 1` (Compound Shard Key)

Decision Logic:

- **Data Locality:** Users typically request historical data for a specific asset (e.g., "Show me the last 30 days of AAPL"). By using a symbol as the prefix of the shard key, MongoDB ensures that all data points for a specific ticker are stored on the same shard (or a contiguous set of chunks);
- **Write Distribution:** Since stock market updates usually occur in batches (e.g., daily closes), using only the date would create a "hotspot" where all writes for a specific day hit a single shard. Adding a symbol distributes the write load across the entire cluster;
- **Efficiency:** This turns chart-generation queries into Targeted Queries, significantly reducing inter-node network traffic.

8.2.3 Transactions: User-Centric Partitioning

Transactions are high-growth data points that require strict consistency and fast retrieval for user dashboards.

Selected Shard Key: `"user_id": 1`

Decision Logic:

- **Core Use Case:** The most frequent operation is a user viewing their own trade history or the `tradeService` validating a new order. Partitioning by `user_id` ensures that all of users' transactions reside on the same shard.
- **Atomic Operations:** This choice is vital for performance during Atomic Savings. When the `user_id` is the shard key, multi-document transactions involving the user's profile and their transactions can often be handled within a single shard, avoiding the overhead of complex distributed transactions.
- **Administrative Trade-off:** While global analytical queries (e.g., "Top-K most traded assets") will require a scatter-gather approach across all shards, this is an acceptable trade-off to prioritize the responsiveness of the live trading engine.

8.2.4 News: Strategic Outlook

Given the current volume of news data, sharding is currently disabled to maintain architectural simplicity. The collection resides on the Primary Shard.

In Real-World Scalability scenario: In a production environment with millions of articles, the recommended shard key is `"sector": 1, "date": -1`.

Financial news consumption is often sector-specific (e.g., "Tech" or "Energy"). Sharding by sector allows the system to serve specialized feeds instantly. The descending date ensures that the most recent news—the most frequently accessed—is efficiently indexed within each partition.

8.2.5 Users and Assets: Metadata Stability

The users and assets collections have relatively low cardinality and slow growth. These are kept as Unsharded Collections on the primary shard. This allows for faster global lookups and simplifies the management.

9 Redis Design

9.1 Redis Architectural Role

In the proposed financial system, Redis is deployed as an in-memory data structure store to bridge the gap between persistent storage and real-time responsiveness. While MongoDB acts as the primary persistent database, Redis functions as a volatile, low-latency layer designed to handle high-velocity data and session-specific states. The integration of Redis addresses three primary technical requirements:

- **Latency Reduction:** Minimizing the Round Trip Time (RTT) for frequently accessed data such as current market prices or asset lists.
- **Throughput Optimization:** Offloading repetitive read queries (e.g., news feeds) from MongoDB.
- **Real-time Volatility:** Managing intraday price samples that are transient by nature and do not require immediate persistence.

Only performance-critical and non-persistent data are stored in Redis. MongoDB remains the authoritative source of truth, and Redis data can be safely discarded and reconstructed if inconsistencies occur.

9.2 Redis Key-Value Structure and Data Mapping

The data in Redis is organized into logical namespaces to ensure clarity and efficient lookup.

9.2.1 User key

To optimize memory utilization, Redis only stores the state of active users. Upon login, the user's financial profile is loaded; upon logout or session timeout, the cache is purged. A TTL of 30 minutes is set for user keys to ensure that data is only retained for active users. The keys are:

- `user:id:cash` (String)-`i` represents available cash(balance);
- `user:id:cash:blocked` (String)-`i` represents cash/balance reserved for pending orders;
- `user:id:portfolio` (Set) -`i` represents a list of owned asset symbols (e.g., "AAPL", "BTC");
- `user:id:port:symbol` (HAsH)-`i` represents "quantity", "bep", "blockedQty" for each asset in the user's portfolio.

9.2.2 Asset and Market Data

Market data is categorized into real-time prices and pre-computed global insights.

The keys are:

- `asset:type:list` (Hash) -`i` Map of all assets of a certain type (Field: symbol, Value: name);
- `asset:symbol:current_price` (String) -`i` The most recent real-time market price;
- `asset:symbol:intraday_prices` (ZSet) -`i` Price history of the day. Score: Timestamp, Member: "Timestamp:Price". The score represents the Unix timestamp, allowing the system to perform efficient range queries for small-scale historical charts (e.g., "last 60 minutes");

- `asset:most_traded` (Hash) -> Stats of the most traded assets from the previous day (Field: `symbol`, Value: `json`). Is a pre-computed daily rank to avoid heavy aggregation pipelines in MongoDB;
- `asset:top_growth` (ZSet) -> Ranking of assets with highest % growth. Score: % change. Is a pre-computed daily rank to avoid heavy aggregation pipelines in MongoDB;
- `asset:worst_decline` (ZSet) -> Ranking of assets with highest % decline. Score: % change (negative). Is a pre-computed daily rank to avoid heavy aggregation pipelines in MongoDB.

9.2.3 News

The news items will be displayed in chronological order and will show only some information, without text. The user can scroll through the news items (load older news items) or filter the news items by category or open a news item to view the text. The front-end information for the latest news items for each category is stored in fast-access memory.

The keys are:

- `news:latest` (ZSet) -> Global index of 10 latest news IDs by timestamp;
- `news:latest:sector:sector` (ZSet) -> Sector-specific index of 10 latest news IDs;
- `news:newsId` (Hash) -> Hash structure containing the title, summary, sector, and timestamp of the news stored in the ZSet. The key of this hash structure is the id of the news.

9.3 Synchronization and Persistence Policies

The system employs a Write-Through strategy combined with Scheduled Flushes to maintain synchronization between Redis and MongoDB.

- **Write-Through:** When a user executes a trade, the change is first persisted in MongoDB. Immediately following a successful commit, the corresponding Redis keys (`cash`, `portfolio`) are updated. In the event of a crash, Redis data is discarded, and the state is reconstructed from MongoDB.
- **Intraday Flush (Asset Prices):** Throughout the trading day, price samples accumulate in Redis. At market close, a batch process aggregates these samples (calculating Open, High, Low, Close), persists the result as a single document in MongoDB's `asset_prices` collection, and clears the Redis ZSET.

9.4 Redis Replication and Fault Tolerance Strategy

To augment system availability and ensure low-latency data access, the caching layer is architected using a Redis Master-Replica configuration. The cluster consists of one Master node and two Secondary replicas, providing a robust mechanism for fault tolerance and workload distribution.

The following principles govern the organization of the Redis caching layer:

- **High Availability and Rapid Failover:** The presence of three Redis instances significantly enhances system resilience. In the event of a failure of the Master node, a rapid failover process is triggered, promoting one of the replicas to Master status. This minimizes downtime and mitigates the risk of losing volatile intraday data that has not yet been persisted to MongoDB.

- Read/Write Splitting based on Data Criticality: The system implements a sophisticated routing logic to balance performance and accuracy:
 - Master Node (Strict Consistency): All write operations and critical reads are strictly routed to the Master node. This includes sensitive financial data such as User Cash Balances (current and available), User Portfolios, and the Latest Asset Prices. Since these values directly influence trade executions, the system cannot tolerate even momentary inconsistencies.
 - Replica Nodes (Eventual Consistency): Non-critical data that can tolerate minor replication lags is served by replicas. This includes the Intraday Price History, Asset Lists, Market Rankings (e.g., Most Traded, Top Growth, Worst Decline), and the News Cache. Offloading these high-volume read operations to replicas prevents bottlenecks on the Master node, ensuring the trading engine remains highly responsive.

10 Database Operations and Queries

10.1 CRUD Operations

The system's interaction with MongoDB follows the standard RESTful paradigm, mapping HTTP methods to Create, Read, Update, and Delete (CRUD) operations. This architectural choice ensures a predictable interface for the frontend while maintaining a clear separation of concerns within the Data Access Object (DAO) layer.

The association between HTTP methods and CRUD operations is as follows:

- GET: Read;
- POST: Create;
- PUT: Update;
- DELETE: Delete;
- PATCH: partial update;
- OPTIONS: HTTP available methods.

The project implements all basic CRUD operations for each MongoDB collection (excluding counters). Detailed documentation for these REST APIs, including schemas and interactive testing, is available via the Swagger UI and the project's GitHub README.

10.2 Advanced Queries and Aggregations

To provide sophisticated financial analytics, the system leverages the MongoDB Aggregation Framework. These pipelines process data in stages to generate high-level insights for both users and administrators.

The aggregations implemented in this system are described below, divided according to the collection on which they act.

```
/**
 * View the 10 users with the largest portfolios in terms of different assets.
 *
 * @return List of the user
 */
Usage: 1 Arzaziel
@Override
public List<UserVarietyDTO> findTop10ByPortfolioVariety() {
    Aggregation aggregation = Aggregation.newAggregation(
        // get user_id, username, size of wallets as 's', 'e' and 'c'
        Aggregation.project(
            ...fields: "user_id", "email"
            .and(ArrayOperators.Size.lengthOfArray(ConditionalOperators.ifNull(
                fieldReference: "shareWallet").then(Collections.emptyList()))).as(
                alias: "s")
            .and(ArrayOperators.Size.lengthOfArray(ConditionalOperators.ifNull(
                fieldReference: "etfWallet").then(Collections.emptyList()))).as(
                alias: "e")
            .and(ArrayOperators.Size.lengthOfArray(ConditionalOperators.ifNull(
                fieldReference: "cryptoWallet").then(Collections.emptyList()))).as(
                alias: "c"),
            // calculate the total sum of the assets (sum of s, e and c) as totalDistinctAssets
            Aggregation.project(
                ...fields: "user_id", "email").and(Expression(
                    expression: "s + e + c").as(
                        alias: "totalDistinctAssets"),
            // sort the result in descending order
            Aggregation.sort(Sort.Direction.DESC,
                ...fields: "totalDistinctAssets"),
            // take the top 10 users
            Aggregation.limit(maxElements: 10),
            // rename the field with the correct name for DTO -> UserVarietyDTO has userId, username, totalDistinctAssets
            Aggregation.project().and(
                name: "user_id").as(
                    alias: "userId")
                .and(
                    name: "email").as(
                        alias: "username").andInclude(
                            ...fieldNames: "totalDistinctAssets"
            );
    return mongoTemplate.aggregate(aggregation,
        collectionName: "users", UserVarietyDTO.class).getMappedResults();
}
```

Figure 34: Image showing the code for the 10 users with the largest portfolios in terms of different assets.

```

/**
 * View the 10 users with the largest amount of a given asset in their portfolio.
 *
 * @param symbol symbol of the asset to looking for
 * @param type the type of the asset, used for identify the correct wallet to look in
 * @return list of the user
 */
! usage  ± Arzazrel
@Override
public List<UserTopAssetHolderDTO> findTop10HoldersByAsset(String symbol, AssetType type) {
    // get the correct name of the wallet to search in based on the AssetType (share, etf, crypto)
    String walletField = type + "Wallet";

    Aggregation aggregation = Aggregation.newAggregation(
        // divide the array of asset
        Aggregation.unwind(walletField),
        // get only the searched asset
        Aggregation.match(Criteria.where("key: " + walletField + ".symbol").is(symbol)),
        // sort the quantity of the asset in descending order
        Aggregation.sort(Sort.Direction.DESC, "fields: " + walletField + ".quantity"),
        // take the top 10 users
        Aggregation.limit(maxElements: 10),
        // rename the field with the correct name for DTO -> UserTopAssetHolderDTO has userId, username, quantity
        Aggregation.project().and("name: " + "user_id").as("alias: " + "userId")
            .and("name: " + "email").as("alias: " + "username")
            .and("name: " + walletField + ".quantity").as("alias: " + "quantity")
    );
    return mongoTemplate.aggregate(aggregation, "collectionName: " + "users", UserTopAssetHolderDTO.class).getMappedResults();
}

```

Figure 35: Image showing the code for the 10 users with the largest amount of a given asset in their portfolio.

```

/**
 * View the average, minimum and maximum number of distinct assets held by users
 *
 * @return list of the user
 */
! usage  ± Arzazrel
@Override
public GlobalUserStatsDTO getGlobalUsageStats() {
    Aggregation aggregation = Aggregation.newAggregation(
        // get the sizes of the wallets as sCount, eCount, cCount
        Aggregation.project()
            .and(ArrayOperators.Size.lengthOfArray(ConditionalOperators.isNull("fieldReference: " + "shareWallet").then(Collections.emptyList()))).as("alias: " + "sCount")
            .and(ArrayOperators.Size.lengthOfArray(ConditionalOperators.isNull("fieldReference: " + "etfWallet").then(Collections.emptyList()))).as("alias: " + "eCount")
            .and(ArrayOperators.Size.lengthOfArray(ConditionalOperators.isNull("fieldReference: " + "cryptoWallet").then(Collections.emptyList()))).as("alias: " + "cCount"),
        // calculate the total asset for each user
        Aggregation.project().andExpression("expression: " + "sCount + eCount + cCount").as("alias: " + "totalAsset"),
        // group all user for final statistics (default by _id)
        Aggregation.group()
            .avg("reference: " + "totalAsset").as("alias: " + "avgDistinctAssets")
            .min("reference: " + "totalAsset").as("alias: " + "minAssetHeld")
            .max("reference: " + "totalAsset").as("alias: " + "maxAssetsHeld")
    );
    // no rename needed -> GlobalUserStatsDTO has avgDistinctAssets, minAssetHeld, maxAssetsHeld
    return mongoTemplate.aggregate(aggregation, "collectionName: " + "Users", GlobalUserStatsDTO.class).getUniqueMappedResult();
}

```

Figure 36: Image showing the code for viewing the average, minimum, and maximum number of distinct assets held by users.

10.2.1 User

The aggregations that operate on the users collection are:

- View the 10 users with the largest portfolios in terms of different assets: A pipeline analyzes the User collection to identify the "Top-10" users by portfolio diversity (number of different assets owned). Image 34;
- View the 10 users with the largest amount of a given asset in their portfolio. Image 35;
- View the average, minimum, and maximum number of distinct assets held by users: aggregation in which the system computes the average, minimum, and maximum number of distinct assets held across the user base. Image 36;

```
/**
 * Number of assets by type (share / ETF / crypto)
 *
 * @return List of AssetTypeCountDTO containing the result for each asset type
 */
1 usage  ▲ Arzazrel
@Override
public List<AssetTypeCountDTO> countAssetsByType() {
    Aggregation aggregation = Aggregation.newAggregation(
        // group and count all asset by asset type
        Aggregation.group( ...fields: "type").count().as( alias: "count"),
        // rename the field with the correct name for DTO -> AssetTypeCountDTO has type, count
        Aggregation.project( ...fields: "count").and( name: "_id").as( alias: "type")
    );

    return mongoTemplate.aggregate(aggregation, collectionName: "assets", AssetTypeCountDTO.class).getMappedResults();
}
```

Figure 37: Image showing the code for the number of assets by type (share / ETF / crypto).

```
/**
 * Top 10 sectors by number of Listed share
 *
 * @return List of SectorShareCountDTO containing the 10 top sectors and their counts
 */
1 usage  ▲ Arzazrel
@Override
public List<SectorShareCountDTO> findTop10SectorsByShares() {
    Aggregation aggregation = Aggregation.newAggregation(
        // filter get only shares (ETFs and cryptocurrencies do not have a sector field)
        Aggregation.match(Criteria.where( key: "type").is(AssetType.share.name())),
        // group and count all share by sector
        Aggregation.group( ...fields: "sector").count().as( alias: "count"),
        // sort the quantity of the asset in descending order
        Aggregation.sort(Sort.Direction.DESC, ...fields: "count"),
        // take the top 10 share sector
        Aggregation.Limit( maxElements: 10),
        // rename the field with the correct name for DTO -> SectorShareCountDTO has sector, count
        Aggregation.project( ...fields: "count").and( name: "_id").as( alias: "sector")
    );

    return mongoTemplate.aggregate(aggregation, collectionName: "assets", SectorShareCountDTO.class).getMappedResults();
}
```

Figure 38: Image showing the code for the top 10 sectors by number of listed shares.

10.2.2 Asset

The aggregations that operate on the asset collection are:

- Number of assets by type (share / ETF / crypto): image 37;
- Top 10 sectors by number of listed shares: image 38.

With this aggregation, assets are aggregated by type (Share, ETF, Crypto) and sector to provide a macroeconomic overview of the platform's listed instruments.

```
* View the top 10 assets with the best growth/worst decline last day/week/month.
*
* @param window    time window considered
* @param ascending if true -> worst , if false -> top
* @return list of AssetGrowthDTO containing the result
*/
4 usages  Arzazrel
@Override
public List<AssetGrowthDTO> findAssetPerformance(TimeWindow window, boolean ascending) {

    Instant startDate = DateUtils.calculateStartDate(window);          // calculate the start date

    Aggregation aggregation = Aggregation.newAggregation(
        // filter get only asset_prices more recent than start time
        Aggregation.match(Criteria.where("date").gte(startDate)),
        // sort the timestamp in ascending order
        Aggregation.sort(Sort.Direction.ASC, "date"),
        // group by symbol and take the first price of the window (open) and the last price of the window (close)
        Aggregation.group("symbol")
            .first("open").as("firstPrice")
            .last("close").as("lastPrice"),
        // calculate percentage: ((lastPrice - firstPrice) / firstPrice) * 100
        Aggregation.project("firstPrice", "lastPrice")
            .andExpression("((lastPrice - firstPrice) / firstPrice * 100)").as("percentageChange"),
        // sort related the 'ascending' parameter
        Aggregation.sort(ascending ? Sort.Direction.ASC : Sort.Direction.DESC, "percentageChange"),
        // take the top/worst
        Aggregation.limit(maxElements: 10),
        // rename the field with the correct name for DTO -> AssetGrowthDTO has symbol, percentageChange, window
        Aggregation.project("percentageChange")
            .and("_id").as("symbol")
            .andExpression("'" + window.name() + "'").as("window")
    );

    return mongoTemplate.aggregate(aggregation, "asset_prices", AssetGrowthDTO.class).getMappedResults();
}
```

Figure 39: Image showing the code for viewing the top 10 assets with the best growth/worst decline over the time window.

```

public List<AssetStableTrendDTO> findConsistentTrendAssets(boolean positiveTrend) {
    Instant oneWeekAgo = Instant.now().minus( amountToSubtract: 7, ChronoUnit.DAYS); // calculate the start date
    Object oppositeTrend = positiveTrend ? false : true; // get opposite trend

    Aggregation aggregation = Aggregation.newAggregation(
        // filter get only asset_prices more recent than start time
        Aggregation.match(Criteria.where( key: "date").gte(oneWeekAgo)),
        // create a Boolean field: 'true' if positive trend, 'false' if negative trend
        Aggregation.project( ...fields: "symbol")
            .andExpression( expression: "(close - open) / open * 100").as( alias: "dailyPriceChange")
            .and(ConditionalOperators.when(ComparisonOperators.Gt.valueOf( fieldReference: "close").greaterThan( fieldReference: "open"))
                .then( value: true).otherwise( value: false)).as( alias: "isPositive"),
        // group by symbol
        Aggregation.group( ...fields: "symbol")
            .push( reference: "isPositive").as( alias: "allDaysTrend") // group isPositive in an array
            .avg( reference: "dailyPriceChange").as( alias: "averageRate")
            .min( reference: "dailyPriceChange").as( alias: "minRate")
            .max( reference: "dailyPriceChange").as( alias: "maxRate"),
        // filter only the symbol with same trend (positiveTrend) for all the window
        Aggregation.match(Criteria.where( key: "allDaysTrend").nin(oppositeTrend)),
        // sort by averageRate
        Aggregation.sort(positiveTrend ? Sort.Direction.DESC : Sort.Direction.ASC, ...fields: "averageRate"),
        // take top 10 doc
        Aggregation.limit( maxElements: 10),
        // rename the field with the correct name for DTO -> AssetStableTrendDTO has symbol, averageRate, minRate, maxRate, positiveTrend
        Aggregation.project( ...fields: "averageRate", "minRate", "maxRate")
            .and( name: "_id").as( alias: "symbol")
            .and(new AggregationExpression() {
                no usages
                Arzazrel
                @Override
                public org.bson.Document toDocument(AggregationOperationContext context) {
                    return new org.bson.Document("$literal", positiveTrend);
                }
            }).as( alias: "positiveTrend")
    );

    return mongoTemplate.aggregate(aggregation, collectionName: "asset_prices", AssetStableTrendDTO.class).getMappedResults();
}

```

Figure 40: Image showing the code for the 10 asset have consistently risen/fell over the past week.

10.2.3 Asset_prices

Aggregations on asset_prices and transactions provide critical market sentiment data:

- View the top 10 assets with the best growth/worst decline over the time window (day, week, month, ...): By comparing historical values (asset_prices collection) the system identifies the best and worst performers over a time window. Compare Open price value of the first day of the time window and the Close price value for the last day of the time window for assets to identify the best and worst performing assets. Image 39;
- See the 10 assets that have consistently risen over the past week (i.e., each day closed higher than its opening price for a week) and their average daily growth rate: Identify "Trend Leaders" that have closed higher than their opening price for seven consecutive days, calculating their average daily growth rate. Image 40;
- See the 10 assets that consistently fell last week (i.e., closed every day negatively compared to their opening for a week) and their average decline loss ratio: Identify "Trend Leaders" that have closed lower than their opening price for seven consecutive days, calculating their average daily decline rate. Image 40.

```

/**
 * Calculate the top 10 most traded assets on the platform for day/week/month/year
 *
 * @param window analysis time window (DAY, WEEK, MONTH, YEAR)
 * @return A list of MostTradedAssetDTO sorted by descending monetary volume.
 */
3 usages  ▴ Arzazrel
@Override
public List<MostTradedAssetDTO> findMostTradedAssets(TimeWindow window) {
    Instant startDate = DateUtils.calculateStartDate(window);           // calculate the start date

    Aggregation aggregation = Aggregation.newAggregation(
        // filter only transaction more recent from the start date
        Aggregation.match(Criteria.where("date").gte(startDate)),
        // filter only transaction of buy or sell
        Aggregation.match(Criteria.where("symbol").ne("value: null")),
        // group by symbol
        Aggregation.group(...fields: "symbol")
            .count().as("transactionCount")           // count number of transaction
            .sum(reference: "quantity").as("totalQuantity") // count the total quantity of asset for all transactions
            .sum(reference: "totalPrice").as("totalVolume"), // sum the total price (quantity * asset price)
        // Sort by decreasing monetary volume
        Aggregation.sort(Sort.Direction.DESC, ...fields: "totalVolume"),
        // take top 10
        Aggregation.limit(maxElements: 10),
        // rename the field with the correct name for DTO -> MostTradedAssetDTO has symbol, transactionCount, totalQuantity, totalVolume
        Aggregation.project(...fields: "transactionCount", "totalQuantity", "totalVolume")
            .and(name: "_id").as("symbol")
    );

    return mongoTemplate.aggregate(aggregation, collectionName: "transactions", MostTradedAssetDTO.class).getMappedResults();
}

```

Figure 41: Image showing the code for the top 10 most traded assets on the platform over the time window (day, week, month, ...).

```

/**
 * Analyze the number of transaction, by categories or by payment type for day/week/month/year.
 * Offering an overview of user payment preferences
 *
 * @param groupByField The field on which to perform the grouping (e.g., "type" or "paymentMethod").
 * @param window analysis time window (DAY, WEEK, MONTH, YEAR)
 * @return A list of TransactionDistributionDTO with counts and volumes for each group.
 */
1 usage  ▴ Arzazrel
@Override
public List<TransactionDistributionDTO> getTransactionDistribution(TransactionGroupField groupByField, TimeWindow window){
    Instant startDate = DateUtils.calculateStartDate(window);           // calculate the start date
    String fieldName = groupByField.name();

    Aggregation aggregation = Aggregation.newAggregation(
        // filter only transaction more recent from the start date
        Aggregation.match(Criteria.where("date").gte(startDate)),
        // group by parameter (es. "paymentMethod")
        Aggregation.group(fieldName)
            .count().as("count")
            .sum(reference: "totalPrice").as("totalAmount"),
        // rename the field with the correct name for DTO -> TransactionDistributionDTO has category, count, totalAmount, timeWindow
        Aggregation.project(...fields: "count", "totalAmount")
            .and(name: "_id").as("category")
            .andExpression(expression: "" + window.name() + "").as("timeWindow")
    );

    return mongoTemplate.aggregate(aggregation, collectionName: "transactions", TransactionDistributionDTO.class).getMappedResults();
}

```

Figure 42: Image showing the code for analyzing the number of transactions, by categories or by payment type, over time.

```

/**
 * Calculates the total capital invested in the purchase of assets during a specific period.
 * The aggregation filters only "BUY" transactions.
 *
 * @param symbol (Optional) The symbol of a specific asset. If null, the calculation is global.
 * @param window The time window for calculating the investment.
 * @return A TotalInvestmentDTO containing the amount invested and the number of purchases.
 */
// usage: Arzazel
@Override
public TotalInvestmentDTO getTotalMoneyInvested(String symbol, TimeWindow window) {
    Instant startDate = DateUtils.calculateStartDate(window); // calculate the start date

    // create dynamic filter: always type = purchase and by data
    Criteria criteria = Criteria.where("transactionType").is("purchase").and("date").gte(startDate);
    // if the symbol is provided, add the filter for the specific asset.
    if (symbol != null && !symbol.isEmpty())
        criteria.and("symbol").is(symbol);

    Aggregation aggregation = Aggregation.newAggregation(
        // match by created filter
        Aggregation.match(criteria),
        // standard group
        Aggregation.group()
            .sum(reference: "totalPrice").as("totalInvested")
            .count().as("numberOfTransactions"), // count number of transactions
        // rename the field with the correct name for DTO -> TotalInvestmentDTO has totalInvested, numberOfTransactions, window
        Aggregation.project(
            fields: "totalInvested", "numberOfTransactions"
            .andExpression(expression: "" + window.name() + "").as("window")
        )
    );

    // run aggregation
    List<TotalInvestmentDTO> results = mongoTemplate.aggregate(aggregation, "transactions", TotalInvestmentDTO.class).getMappedResults();
    // If there are no transactions in the period, we return an empty DTO or one with zeros.
    return results.isEmpty() ? new TotalInvestmentDTO(totalInvested: 0.0, numberOfTransactions: 0L, window) : results.get(0);
}

```

Figure 43: Image showing the code for viewing the total amount of money invested in assets over a specific time range (e.g, week, month, ...).

```

/**
 * Rank users based on their net financial flow. The flow is calculated as: (Total Sales) - (Total Purchases).
 * A positive value indicates a user who is liquidating positions (cashing out),
 * A negative value indicates a user who is accumulating assets (investing capital).
 *
 * @param window The analysis time window.
 * @param ascending If true, returns users with the worst flow (those who invested the most).
 *                  If false, returns users with the best flow (those who earned the most).
 * @return A List of UserFinancialFlowDTO with details of volumes per user.
 */
// usage: Arzazel
@Override
public List<UserFinancialFlowDTO> findUserFinancialFlow(TimeWindow window, boolean ascending){
    Instant startDate = DateUtils.calculateStartDate(window); // calculate the start date

    Aggregation aggregation = Aggregation.newAggregation(
        // filter only transaction more recent from the start date
        Aggregation.match(Criteria.where("date").gte(startDate)),
        // conditional flow: if BUY -> totalBought, if SELL -> in totalSold
        Aggregation.project(
            fields: "user_id", "totalPrice", "transactionType"
            .and(ConditionalOperators.when(Criteria.where("transactionType").is("purchase"))
                .then(Fields.field("totalPrice")).otherwise(value: 0.0)).as("buyAmount") // sum in buy contributor
            .and(ConditionalOperators.when(Criteria.where("transactionType").is("sell"))
                .then(Fields.field("totalPrice")).otherwise(value: 0.0)).as("sellAmount"), // sum in sell contributor
        )
        // group by user id
        Aggregation.group(
            fields: "user_id"
            .sum(reference: "buyAmount").as("totalBought")
            .sum(reference: "sellAmount").as("totalSold"),
        )
        // calculate Net Flow: (Sold - Bought)
        Aggregation.project(
            fields: "totalBought", "totalSold"
            .and(name: "_id").as("userId")
            .andExpression(expression: "totalSold - totalBought").as("netFlow"),
        )
        // rename the field with the correct name for DTO -> UserFinancialFlowDTO has userId, netFlow, totalBought, totalSold
        Aggregation.project(
            fields: "userId", "netFlow", "totalBought", "totalSold",
        )
        // order by netFlow
        Aggregation.sort(ascending ? Sort.Direction.ASC : Sort.Direction.DESC, fields: "netFlow"),
        // take the top 10
        Aggregation.limit(maxElements: 10)
    );

    return mongoTemplate.aggregate(aggregation, "transactions", UserFinancialFlowDTO.class).getMappedResults();
}

```

Figure 44: Image showing the code for viewing the Top-K users with the highest/worst net financial flow generated by buy and sell operations within a selected time window (e.g, week, month, ...).

```

/**
 * Calculates the total monetary volume (sum of totalPrice) for each asset traded during the current day.
 *
 * @return A list of DailyVolumeDTO containing the symbol and its total volume.
 */
1 usage 1 Arzazrel
@Override
public List<DailyVolumeDTO> getDailyVolumeBySymbol() {
    // want the volume of all assets from the start of the current day (00:00)
    Instant startOfDayToday = LocalDate.now()
        .atStartOfDay(ZoneId.of(ZoneId.of("America/New_York"))) // use NY tie zone for project specific
        .toInstant();

    Aggregation aggregation = Aggregation.newAggregation(
        // filter only transactions from today that have a symbol (exclude cash-only transfers)
        Aggregation.match(Criteria.where("date").gte(startOfDayToday).and("symbol").ne("value: null")),
        // group by symbol and sum the totalPrice
        Aggregation.group("symbol")
            .sum("totalPrice").as("volume"),
        // project to DTO fields
        Aggregation.project("volume")
            .and("_id").as("symbol")
    );

    return mongoTemplate.aggregate(aggregation, "transactions", DailyVolumeDTO.class).getMappedResults();
}

```

Figure 45: Image showing the code for getting the total monetary volume (sum of totalPrice) for each asset traded during the current day.

10.2.4 Transaction

- View the top 10 most traded assets on the platform over the time window (day, week, month, ...). Image 41;
- Analyze the number of transaction, by categories or by payment type, over time. Offering an overview of user payment preferences. Image 42;
- View the total amount of money invested in assets over a specific time range (e.g, week, month, ...). Sum of totalPrice of all buy transactions within a given time window. Very interesting for admin analytics. Image 43;
- View the Top-K users with the highest/worst net financial flow generated by buy and sell operations within a selected time window (e.g., week, month, ...). Buy operations contribute negatively to the flow, while sell operations contribute positively. Users are ranked in descending order based on their net financial impact. When looking at the worst net financial users, this aggregation highlights users who have invested the most capital in asset purchases relative to their selling operations. Image 44;
- Calculates the total monetary volume (sum of totalPrice) for each asset traded during the current day. It is used to calculate the volume for the final-day aggregation of daily asset prices. Image 45.

These aggregations provide a comprehensive view of capital movement within the platform.

```

/**
 * Counts how many news articles were published for each sector in a given window.
 * Useful for identifying which market areas are currently most active.
 *
 * @param window time window considered
 */
1 usage  1 Arzazrel
@Override
public List<SectorNewsCountDTO> countNewsBySector(TimeWindow window) {
    Instant startDate = DateUtils.calculateStartDate(window); // calculate start date

    Aggregation aggregation = Aggregation.newAggregation(
        // filter get only asset_prices more recent than start time
        Aggregation.match(Criteria.where("date").gte(startDate)),
        // group by sector and count the news for each sector
        Aggregation.group(...fields: "sector").count().as( alias: "newsCount"),
        // sort by count news
        Aggregation.sort(Sort.Direction.DESC, ...fields: "newsCount"),
        // rename the field with the correct name for DTO -> SectorNewsCountDTO has sector, newsCount, window
        Aggregation.project(...fields: "newsCount")
            .and( name: "_id").as( alias: "sector")
            .andExpression( expression: "" + window.name() + "").as( alias: "window")
    );

    return mongoTemplate.aggregate(aggregation, collectionName: "news", SectorNewsCountDTO.class).getMappedResults();
}

```

Figure 46: Image showing the code to see the number of news per sector in a time window.

```

/**
 * Identifies the top 5 assets (symbols) most frequently mentioned in news articles.
 * Helps track which companies are "trending" in the media.
 *
 * @param window time window considered
 */
1 usage  1 Arzazrel
@Override
public List<TopMentionedAssetDTO> findTopMentionedAssets(TimeWindow window) {
    Instant startDate = DateUtils.calculateStartDate(window);

    Aggregation aggregation = Aggregation.newAggregation(
        // filter get only asset_prices more recent than start time
        Aggregation.match(Criteria.where("date").gte(startDate).and( key: "company").exists( value: true).ne( value: null)),
        // group by company name and count
        Aggregation.group(...fields: "company").count().as( alias: "mentionCount"),
        // sort by mentionCount
        Aggregation.sort(Sort.Direction.DESC, ...fields: "mentionCount"),
        // take the top 5
        Aggregation.limit( maxElements: 5),
        // rename the field with the correct name for DTO -> TopMentionedAssetDTO has companyName, mentionCount, window
        Aggregation.project(...fields: "mentionCount")
            .and( name: "_id").as( alias: "companyName")
            .andExpression( expression: "" + window.name() + "").as( alias: "window")
    );

    return mongoTemplate.aggregate(aggregation, collectionName: "news", TopMentionedAssetDTO.class).getMappedResults();
}

```

Figure 47: Image showing the code for identifying the top 5 companies that have been mentioned in the highest number of news articles within a given time window.

10.2.5 News

- See the number of news per sector in a time window. This is useful to quantify news density per sector within specific periods. Image 46;
- Identify the top 5 companies that have been mentioned in the highest number of news articles within a given time window. This can be useful to aiding in sentiment-driven trading decisions. Image 47.

10.3 Operation interacting with Redis Caching

Below is a list of operations, divided according to the collection on which they operate, that interact with the Redis cache.

- Assets
 - Asset Creation: The new asset is appended to the global asset list filtered by its category (Shares, ETFs, or Cryptocurrencies) in the `asset:type:list` Hash;
 - Asset Deletion (Soft Delete):
 - * Removal from the corresponding `asset:type:list`;
 - * Eviction of the `asset:symbol:current_price` key;
 - * Processing and subsequent deletion of all associated `asset:symbol:intraday_prices` time-series data;
 - * Removal of the asset from all ranking structures, including `most_traded`, `top_growth`, and `worst_decline`.
 - Asset Reactivation: Re-insertion into the appropriate `asset:type:list` to ensure visibility in the trading dashboard.
- Users
 - Registration & Login: User profile data (cash balance and portfolio composition) is loaded from MongoDB into Redis to facilitate low-latency operations;
 - Logout: eviction of all user-related keys from Redis to optimize memory consumption;
 - Cache Heartbeat (Read/Update of a key): Every access to a user-specific key triggers an automated TTL (Time-To-Live) update, ensuring active sessions remain in memory while idle data is eventually purged;
 - Trade Execution: Upon completion of a transaction, the cached user state (cash and assets) is either updated or reset to prevent inconsistencies between the balance shown to the user and the primary database;
 - Account Suspension/Deletion: Immediate cache flush for the specific `userId` to prevent any further unauthorized operations;
- News
 - News Ingestion
 - * Update the global `news:latest Sorted Set`;
 - * Update the sector-specific `news:latest:sector:sector Sorted Set`;
 - * Create the detailed `news:id` Hash entry.
 - News Deletion: The system performs a membership check across all latest-news lists; if the ID is present, it is removed to ensure the feed remains up to date.

10.3.1 Read Optimization (Caching Strategy)

For high-traffic GET requests, for information stored in the Redis cache, the APIs first check whether the information is present in Redis (to provide data quickly) and, in the event of a cache miss, they will retrieve the information from MongoDB and update the Redis cache. Below are examples of cases where reads interact with the Redis cache:

- **News Feeds:** When requesting the latest news via `/api/users/news/latest`, the system first queries Redis (`news:latest ZSET`). If the requested range (offset/limit) falls within the cached subset (e.g., the first 10 items), the result is served in sub-millisecond time. A cache miss triggers a fallback to MongoDB, which then repopulates the cache;
- **User Financial State:** Requests for `/api/customers/me/cash` or `/me/portfolio` are served directly from Redis for active sessions. This ensures that the real-time dashboard remains responsive even during periods of high market volatility;
- **Assets retrieval:** When users request to view the list of assets available per system or the main daily statistics on assets, this information will be taken from the Redis cache.

10.3.2 Write-Through and Cache Invalidation

To maintain synchronization, write operations must manage both storage layers:

- **Market Updates:** When an administrator adds a new asset or price, the system updates MongoDB and simultaneously refreshes the `asset:type:list` or `asset:symbol:current_price` in Redis;
- **Trade Execution:** Upon a successful trade transaction in MongoDB, the system immediately updates the user's cached cash balance and portfolio composition in Redis;
- **Intraday prices aggregation:** Intraday price samples are collected in Redis throughout the trading session. At the end of the day, a batch process aggregates these ZSETs and performs a bulk `insert_many` into MongoDB's `asset_prices` collection, followed by a Redis flush to prepare for the next trading day.

11 Considerations on CAP Theorem

Analyzing the system requirements, it becomes evident that a financial trading platform cannot operate under a "Total AP" model. Unlike social networking applications, where temporary inconsistencies in "likes" or "posts" are tolerable, a trading system managing real-world assets and cash balances must guarantee that every transaction is accurate, durable, and synchronized. Consequently, our architecture is strategically positioned in the CP (Consistency and Partition Tolerance) edge of the CAP triangle for all transactional operations.

11.1 Emphasis on Consistency in the System's Architecture

In the architectural design of MyFuture, I have deliberately chosen to prioritize Consistency (C) and Partition Tolerance (P). This decision is critical because financial integrity is the core value of the application. By focusing on CP, we ensure that the system prevents "double-spending" or "stale-price" scenarios. If a network partition occurs and the system cannot guarantee that a majority of nodes are synchronized, it will prioritize data correctness over availability for write operations. This ensures that no user can execute a trade based on outdated balance information, preserving the reliability of the entire financial ecosystem.

11.2 Scalability and High Availability Strategy

Despite the emphasis on consistency, the system is designed to be robust and highly available through a carefully configured MongoDB Replica Set and a Redis Master-Replica cluster. To ensure high availability without sacrificing financial accuracy, we have organized our nodes as follows:

- **Primary Node (MongoDB Master):** In the current implementation, this node handles all incoming write requests and all read operations. By directing all traffic to the Primary, we ensure a "Single Source of Truth" and absolute consistency. This setup eliminates any risk of data conflicts or stale reads during the initial phase, as the primary node manages the authoritative state of user wallets, asset ledgers, and market data.
- **Secondary Nodes (MongoDB Replicas):** These nodes provide redundancy and ensure data persistence. In the event of a primary node failure, a new primary is elected among the secondaries through the majority-based consensus. The configuration uses `w=majority`, meaning a trade is only confirmed if at least two nodes have recorded the data, ensuring durability even during node failovers.

11.3 Balancing Consistency and Availability: The Hybrid Model

The proposed architectural strategy implements a nuanced trade-off through a Hybrid Consistency Model, which differentiates between sensitive financial records and general informational data. This is primarily achieved through the strategic use of the Redis caching layer:

- **Strict Consistency (CP) for Transactional Data:** For critical fields such as `user cash`, `user portfolio`, and `current prices for assets`, the system enforces **Master-only** access. By routing these requests exclusively to the Redis Master node, we eliminate the "stale read" problem inherent in asynchronous replication. This ensures that users always interact with their actual, real-time financial state, which is vital for the integrity of the trading engine.

- **Eventual Consistency (AP) for Market Insights:** To maximize throughput and responsiveness, non-critical data—such as news headlines, market rankings (e.g., Best/Worst performers), and historical price charts—is served via Redis Replicas. In this scenario, we accept millisecond-level eventual consistency as a calculated trade-off to ensure the user interface remains fluid and highly available, even during periods of high market volatility.

This dual-track approach underlines a commitment to providing a system that is both resilient and uncompromising regarding data integrity. By meticulously balancing the load and differentiating data criticality, the architecture protects user assets (CP) while maintaining the high performance (AP) expected from a modern, data-intensive trading platform.

11.4 Design for Future Read-Write Splitting

While the system currently enforces a strict Primary-read policy for all MongoDB collections to prioritize financial integrity, the architecture is designed to support Read-Write Splitting. Data entities have been categorized by criticality:

- **Critical (Transactional):** User balances and Trade orders (transactions), which will always require Primary access.
- **Non-Critical (Informational):** Asset lists, historical price samples, and news.

Future iterations will offload the latter to Secondary Nodes using the `secondaryPreferred` read preference (in MongoDB). This will significantly increase the system's throughput by distributing the query load across the entire replica set, accepting a negligible replication lag in exchange for superior performance.

12 Implementation Details

12.1 Frameworks and Technologies

The system is developed following a RESTful architecture, leveraging industry-leading technologies to ensure scalability, security, and high performance.

- **Spring Boot:** The core framework used to create the stand-alone application. It simplifies configuration and deployment through the broader Spring ecosystem and its "convention over configuration" approach;
- **Spring Data MongoDB:** Utilized for seamless interaction with the MongoDB document-oriented database. It provides high-level abstractions like `MongoTemplate` for executing complex queries and native aggregation pipelines;
- **Spring Data Redis (Lettuce):** The driver employed for managing the in-memory cache. It is configured in a Master-Replica setup to optimize workload distribution: the Master node handles all writes, and critical consistency reads (e.g., cash balance), while Replicas serve non-critical data (e.g., news feeds).;
- **Spring Security & JWT:** Implemented for robust authentication and authorization. The system uses JSON Web Tokens (JWT) to intercept requests, validate user identity, and extract roles and identifiers from the `SecurityContext`;
- **Lombok:** A library used to minimize boilerplate code. It automatically generates getters, setters, constructors, and other utility methods at compile-time through annotations such as `@Data`.

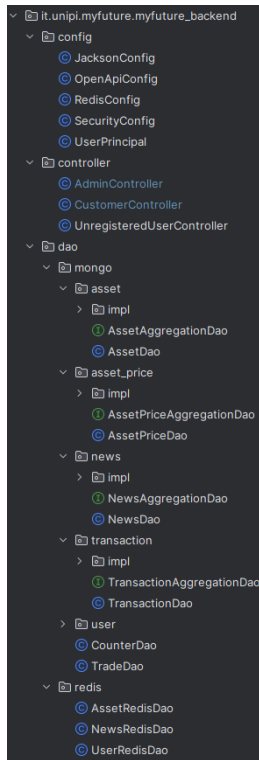
12.2 Project Structure and Packages

The project adopts a strict layered architecture to ensure component decoupling and ease of maintenance. The package subdivision reflects this hierarchy:

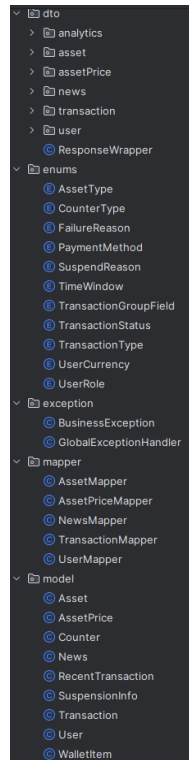
- **Controller:** The entry point of the application. It exposes REST API endpoints and manages basic input validation. It returns `ResponseEntity` objects that encapsulate DTOs, ensuring standardized HTTP responses. In this system, there is a controller for each system's actor (unregistered user, registered user, and admin);
- **Service:** This layer contains the business logic and is divided into interfaces and their respective implementations (e.g., `UserServiceImpl`). It coordinates interactions between different DAOs (MongoDB and Redis) to ensure data consistency across storage layers;
 - **MarketSchedulerService:** A scheduling service (based on `@Scheduled`) responsible for periodic data synchronization. It handles price retrieval from external APIs and the nightly "flush" of intraday prices from Redis to MongoDB;
 - **LimitOrderExecutionServiceImpl:** Implements the logic for executing limit orders (pending orders) when the market opens;
 - **DepositWithdrawalServiceImpl:** Exclusively manages monetary flow operations (deposits and withdrawals). It updates the cash balance on both MongoDB (for persistence) and Redis (for immediate availability), guaranteeing the integrity of the user's financial portfolio;

- TradeService: A cross-cutting service (orchestrator) that manages "market" buy/sell processes involving multiple entities and databases.
- This layer is responsible for data transformation between the different levels of the architecture;
- DAO (Data Access Object): A dedicated layer for physical data access, further separated by technology. It is divided according to the database on which it operates, in our system or Mongo.
 - dao.mongo: Manages primary persistence in MongoDB. Standard queries use dedicated repository classes, while aggregations follow an Interface + Impl pattern to isolate complex computational data-mining logic;
 - dao.redis: Handles volatile data structures in memory, managing TTL (Time-To-Live) policies and caching logic.
- model (or entity): Defines the structure of the documents stored in the database. These classes are persistence-oriented and are never exposed directly via REST to prevent the accidental disclosure of sensitive fields (e.g., password hashes);
- DTO (Data Transfer Object): Specialized objects for data exchange with the client (RequestDTO and ResponseDTO). They act as a stable contract between the frontend and backend, isolating the API from structural changes in the database.
- security: Contains security configurations and the implementation of UserPrincipal. The latter is a custom version of UserDetails that includes the user's MongoDB identifier to facilitate cache operations and session tracking;
- exception: Features a GlobalExceptionHandler (annotated with @RestControllerAdvice) that intercepts application exceptions and translates them into consistent, user-friendly HTTP responses.

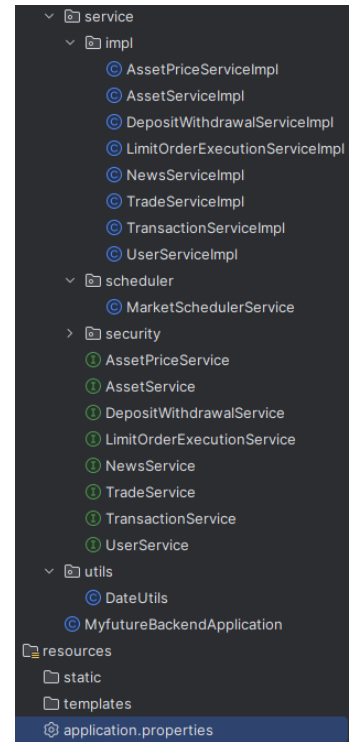
This project structure is shown in image 48.



(a) Image shows config, controller, dao.



(b) Image shows DTO, enums, exception, mapper, model.



(c) Image shows service and utilities.

Figure 48: Architecture Overview: High-level view of the backend components.

13 Use of Artificial Intelligence Tools

13.1 Purpose

Artificial Intelligence (AI) tools, specifically Large Language Models (LLMs) such as Gemini and ChatGPT, were employed throughout the project to optimize the development workflow. The primary objective was to speed up non-decisional tasks, such as technical writing and formal consistency, rather than replacing the architectural decision-making process.

Specifically, these tools were utilized for:

- Code Review and Debugging: identifying syntax errors and logical inconsistencies within the implemented source code.
- Troubleshooting: resolving configuration issues related to the software environment and development tools.
- Empirical Research: gathering real-world system data to compare implementation techniques and refine system requirements.
- Documentation Research: searching for official documentation and best practices regarding specific methods and libraries.
- Repository Optimization: refactoring GitHub repository structures and naming conventions to align with industry standards, thereby enhancing readability and accessibility.
- Translation and Formatting: translation of documentation into academic English.

13.2 How

The interaction with these LLMs involved iterative prompting. Depending on the specific task, snippets of source code, directory structures, or conceptual reflections were provided to the models. This allowed for context-aware suggestions that assisted in navigating complex implementation phases and technical documentation.

13.3 Critical Evaluation

It is essential to state that the use of AI was subject to rigorous human oversight. While several code segments and documentation sections were refined based on AI suggestions, no output was integrated without thorough validation. Every suggestion provided by the AI was critically analyzed and cross-referenced with official documentation.

Consequently, only the components that aligned with the project's specific objectives and architectural requirements were adopted. In many instances, the AI-generated content was significantly modified or rejected entirely to ensure the integrity, security, and originality of the final system. The final implementation remains a direct result of my own technical choices and critical synthesis.

14 Future Improvements and Extensions

14.1 Machine Learning for Financial Insights

The vast amount of historical data stored in the `asset_prices` collection provides a robust foundation for predictive analytics. Future iterations could integrate Machine Learning (ML) models

to provide:

- **Sentiment Analysis:** Leveraging the News collection to correlate news headlines with price movements, providing users with a "Market Sentiment" indicator;
- **Price Forecasting:** Utilizing Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks to suggest potential future price trends based on historical volatility.

14.2 Enhanced Security and Multi-Factor Authentication (MFA)

To reach production-grade security standards, the authentication layer could be extended beyond JWT. Implementing Multi-Factor Authentication (MFA) and OAuth2/OpenID Connect would provide an additional layer of protection for user funds. Furthermore, a more granular Role-Based Access Control (RBAC) system could be implemented to manage different levels of administrative privileges.

14.3 Horizontal Scaling and Microservices Decomposition

As the user base grows, the current monolithic Spring Boot structure could be decomposed into smaller, independent microservices (e.g., a dedicated OrderService, PriceIngestionService, and NewsService). This would allow for independent scaling of components. For instance, the price ingestion service could be scaled horizontally during high market volatility without affecting the performance of the user management system.

14.4 Granular Read Preference Optimization

To further optimize the platform's performance, a granular approach to MongoDB read preferences will be implemented. Currently, the system operates in a full CP mode (Consistency-Priority). By refactoring the Service Layer to use specialized MongoTemplate configurations or @ReadPreference annotations, we can redirect non-transactional queries (such as getAllAssets() or getNews()) to secondary replicas.

This improvement is particularly effective when combined with the existing Redis Cache. Since Redis already provides high-speed access to the latest market data fetched from the Master, the secondary MongoDB nodes can be utilized to serve complex aggregation queries and historical data analysis without impacting the performance of the main trading engine.